VLM-Verified Counterfactual Hindsight for Sparse-Reward Manipulation

Daniel Grant **Parshawn Gerafian** **Matei Gardea**
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
{dgrant,pgerafian,mgardea}@berkeley.edu

Abstract

Sparse-reward manipulation is a credit-assignment problem: TD-signal decays exponentially away from the terminal reward bit, leaving the bulk of every trajectory effectively unprioritized. We make two contributions. **First**, we recast VLM-guided experience replay as *importance-sampled posterior reweighting*: a frozen VLM emits an approximate posterior $q_\phi(t^\star \mid \tau)$ over which timestep caused an episode's outcome, and Semantic PER multiplies the standard TD-error priority by this posterior. The multiplicative form uniquely preserves PER's IS-correction; the additive mixture of the concurrent VLM-RB [28] implicitly retargets the loss. **Second**, we introduce **VLM-Verified Counterfactual Hindsight**, which closes the dominant failure mode of language-only counterfactuals: rather than asking for a hindsight goal (which teleport-collapses in 100% of Opus 4.7 calls on FetchPush), we ask for a corrective action sequence and execute it in a simulator fork. Only simulator-verified relabels enter the buffer (confidence 1.0, zero modeling error). A three-family prompt sweep (GPT-4o / Opus 4.7 / Sonnet 4.5) on 16 Fetch episodes identifies `(GPT-4o, achieved_goal + 5 cm gate)` as strictly dominant (0% collapse, 0.79 plausibility, 0.83 goal-progress), and a 12-episode cross-task pilot achieves 12/12 annotations across Push, PickAndPlace, and Slide with a single template—evidence that a foundation-VLM credit-assignment oracle generalizes where a learned priority head would not.

1 Introduction

Sparse-reward manipulation in environments such as Gymnasium-Robotics Fetch is defined by long horizons (50 steps), one bit of terminal reward, and a goal-achieved threshold of 5 cm. Standard prioritized experience replay (PER) [27] sharpens the sampling distribution toward transitions with large TD-error, but TD-error itself decays exponentially away from the goal-achievement timestep. Hindsight Experience Replay (HER) [1] partially repairs this by relabeling failures as successes for an alternative achieved goal, but the relabel target is constrained to the agent's actually-realized trajectory.

Recent work has explored using foundation Vision-Language Models (VLMs) as "oracles" inside the RL loop, both for reward shaping [25, 15, 30] and, very recently, for replay-buffer prioritization [28]. We argue these uses are best framed not as engineering recipes but as *importance-sampled stochastic optimization with a foundation-model proposal distribution*: the VLM is a frozen approximation to the (intractable) posterior $p(t^\star \mid \tau)$ over which transitions caused an episode's outcome.

This paper makes four contributions.

(1) An IS-posterior framing of VLM-guided replay (Section 3). We re-derive prioritized DQN [27], retrace [19], V-trace [7], and Sharony et al.'s VLM-RB [28] inside a single off-policy-correction

Preprint. Under review.

taxonomy, locating our scheme as *proposal-shaping* with an exogenous foundation-model credit-assignment oracle. The multiplicative form of our update is the *unique* proposal modification (up to trajectory-conditional normalization) that preserves the standard PER importance-sampling correction; the additive mixture used by VLM-RB [28] implicitly retargets the loss to a λ-weighted objective.

(2) Semantic PER with a failure-direction signal (Section 4.1). Sharony et al. score 32-frame clips for goal-satisfaction; we ask the VLM the strictly more informative question—*which single timestep made this trajectory fail?*—and multiply the standard PER priority by a bounded boost over a window around that timestep. The scheme degenerates gracefully to PER when the VLM is uncommitted ($w_{\max} = 1$), needs no λ-warm-up, and admits an unbiased IS-corrected interpretation. Crucially, unlike VLM-as-reward approaches [25, 34, 21] that splice the VLM signal into the TD target, our scheme leaves the target untouched: the env's true sparse reward is recomputed on every sampled transition. VLM miscalibration costs variance in the sampling distribution, not bias in the value function.

(3) VLM-Verified Counterfactual Hindsight (Section 4.3). A direct prompt for an alternative *achieved goal* elicits a teleport-collapse failure mode in 100% of Claude Opus 4.7 calls on FetchPush: the VLM returns `corrective_position` = `desired_goal`, a degenerate zero-information output. We close this loophole *by construction* by asking instead for a corrective *action sequence* and executing it in a fork of the training simulator from the failure-timestep state. A four-vector action cannot teleport a 50 g block by 50 cm under MuJoCo dynamics; the failure mode is structurally inexpressible. The mechanism instantiates the *generator–verifier* paradigm [36] in robot RL—the VLM generates, the simulator verifies—and only trajectories whose sparse reward fires under the unmodified environment dynamics enter the buffer, with confidence 1.0 and zero modeling error.

(4) Empirical analysis: prompt design, VLM-family robustness, and cross-task transfer (Sections 5.3–5.5). A head-to-head GPT-4o vs. Claude Opus 4.7 sweep on six synthetic Fetch failures identifies a strictly-dominating configuration: `(GPT-4o, achieved_goal)` eliminates teleport-collapse (0/6 vs. Opus's 4/4), ties for highest plausibility (0.79), and trails only Opus's degenerate goal-copying on goal-progress (0.83 vs. 0.97). A second prompt sweep on ten *real* SAC-failure episodes confirms the prompt-architectural fix transfers to a third VLM family (Sonnet 4.5: 75% $\to$ 0% teleport on PickAndPlace). A 12-episode cross-task pilot achieves 12/12 task-relevant annotations on Push/PickAndPlace/Slide with a single prompt template, evidence that a foundation-VLM credit-assignment oracle generalizes across structurally-different manipulation tasks where any learned priority head would require per-task fitting.

Section 5 reports SAC+HER baselines on the three Fetch environments, the prompt-design analysis, cross-task transfer evidence, real-data validation, and the current status of two in-flight experiments (HER baselines and a kill-test of the verified-counterfactual mechanism). Section 6 discusses a pipeline-bug discovery that re-frames our headline ablation, small-n caveats, and limitations.

2 Related Work

HER and goal-conditioned RL. Hindsight Experience Replay [1] is the dominant solution to sparse-reward goal-conditioned manipulation; the `future` relabel strategy with $k = 4$ remains a strong baseline a decade later. HER's core move is to convert a failed trajectory τ into a positive demonstration for some achieved goal $g' \in \{\mathrm{ag}_t\}_{t=0}^{T}$ on the trajectory itself, exploiting the fact that the sparse reward function $r(s, a, g) = -\mathbb{1}[\|\mathrm{ag} - g\| > d_{\mathrm{th}}]$ admits cheap re-evaluation under a swapped goal. We organize the recent HER-family literature along three orthogonal extensions of this move—the *relabel target*, the *relabel proposal*, and the *relabel verification step*—and place our work in the third. **Relabel-target extensions** change *which* alternative goals are considered. Single-step Next-Future [20] restricts relabel candidates to ag_{t+1} for sample-efficiency. Contact-Energy Hindsight Prioritization [26] uses privileged end-effector/object state to upweight transitions whose contact-energy correlates with goal achievement—a state-geometry analogue of our heuristic Oracle v3 (§4.1). **Relabel-proposal extensions** change *how* alternative goals or trajectories are synthesized rather than read off the agent's own data. Causal Action Influence Counterfactual Augmentation [29] generates additional transitions on Fetch and Franka-Kitchen by counterfactually intervening on object positions under a learned causal-influence proxy. The HER-for-LM-agents line [12, 5] re-writes failed trajectories as language-instruction relabels, with AgentHER explicitly adding multi-judge

VLM verification to reduce relabel noise (5.9% $\rightarrow$ 2.3%), an early acknowledgment that language-only counterfactuals require an external acceptance test. **Relabel-verification extensions** add an explicit gate between the proposed relabel and the buffer. HInt/NCII [4] asks "would the target object's dynamics change if the cause object were removed?" using a *learned* dynamics model as the verifier; GCHR [16] regularizes the relabel proposal via a hindsight-conditioned policy term. Our verified-counterfactual mechanism (§4.3) sits in this third family but verifies in the *exact training simulator* (zero modeling error) and verifies an *action sequence* rather than a hindsight goal—a stricter test because it requires physics-consistent execution under the original dynamics, not merely a dynamics-model consistency check.

Prioritized replay and off-policy correction. Prioritized DQN [27] samples in proportion to $(|\delta| + \varepsilon)^\alpha$ and reweights every sampled gradient by the annealed importance-sampling correction $w_{\text{IS}} = (|\mathcal{D}| \cdot \mu_P)^{-\beta}$, $\beta\colon \beta_0 \to 1$. The IS correction is the load-bearing structural property of PER: in the $\beta = 1$ limit the reweighted gradient is an unbiased estimator of the uniform-replay loss $\mathbb{E}_U[\ell]$ under self-normalized importance sampling, with the proposal μ_P supplying variance reduction [27, Sec. 3.4]. Multi-step off-policy correction generalizes this idea: Retrace [19] clips per-step IS ratios to bound the trace's variance under truncation-induced bias; V-trace [7] doubly-clips the target and behavior ratios in a distributed actor-learner setup. In each case the proposal is a fixed behavior or policy distribution; the correction inherits its form from the proposal–target ratio. The PER lineage in 2024–2026 has explored *auxiliary* proposal signals beyond raw TD-error while preserving the multiplicative "$\mu_P \cdot g$" structure: ReaPER multiplies the PER priority by a per-transition reliability estimate [24]; RPE-PER replaces $|\delta|^\alpha$ with a reward-prediction-error proposal [35]; Prioritized Generative Replay multiplies the priority by a generative-model relevance score (curiosity or value) on a learned synthetic-experience stream [31]; the non-uniform-memory line shows that almost any non-uniform proposal beats uniform under modest conditions [14]. Ma et al. [17] address a different failure mode—priority staleness when the policy itself is a rapidly-evolving LLM/VLM—via age-decay weights. Within this taxonomy our Semantic PER (§4.1) is a *proposal-shaping* scheme whose auxiliary signal w_{sem} (i) is *exogenous* (a frozen foundation-VLM posterior, not a critic-internal estimate), (ii) is *trajectory-level* (one VLM call per failed episode, then a kernel-smeared weight applied to every slot in that episode), and (iii) combines *multiplicatively* with μ_P, which is the family choice that preserves PER's IS-correction interpretation as derived in §3. The contrasting design choice in the very recent VLM-RB [28] is an *additive* mixture $\lambda\mu_{\text{VLM}} + (1 - \lambda)\mu_U$; we return to this contrast in Table 2 and in Section 4.1.

Foundation models in RL. VLMs have been used as reward sources [25, 30, 32], affordance-guided shaping signals [15], promptable feature encoders [3], counterfactual data labelers [9], token-level counterfactual critics [8], hindsight rewriters for LM agents [12, 5], and online per-timestep reward models for manipulation [34, 21]. Duan et al. [6] fine-tune a VLM (AHA) on synthetic robotic failures and produce natural-language failure explanations that downstream-refine LLM-generated reward code (Eureka pipeline); our system differs in three orthogonal ways. (i) AHA *describes* a failure; we *localize* it to a single timestep usable as a credit-assignment signal. (ii) AHA proposes no corrective actions; we do, and gate them on simulator verification. (iii) AHA's RL integration is reward-refinement (modifies the TD target); ours is replay-prioritization (modifies only the sampling distribution). An AHA-fine-tuned VLM could plausibly replace zero-shot GPT-4o inside our protocol. Foundation-model credit assignment is itself a young thread: Khandoga et al. [13] mask reasoning spans in an LLM policy to identify causally-influential tokens, in the lineage of Mesnard et al. [18] and the credit-assignment survey by Pignatelli et al. [22]. Concurrently, Pignatelli et al. [23] introduce CALM, which uses an LLM to decompose tasks into subgoals and evaluate their achievement zero-shot, providing shaped credit in text-based environments without hand-designed rewards—an independent instantiation of the FM-as-credit-oracle idea in language game settings.

Generator–verifier and verified counterfactual reasoning. Our verification protocol (Section 4.3) sits inside the *generator–verifier* paradigm now popular in LLM reasoning: the generator emits candidates, a verifier (learned or symbolic) accepts the correct ones [36]. We instantiate this in robotics with a foundation-VLM generator and a *symbolic* verifier—the unmodified training simulator—whose decisions are correct by construction. The most related prior work in robotics is the line on counterfactual data augmentation for goal-conditioned RL [29, 4, 9]: each uses a learned proxy (causal-action-influence, learned dynamics, VLM plausibility) to produce additional trajectories. We

Table 1: VLM-Guided Experience Replay (Sharony et al. 2026) vs. this paper.

Axis	VLM-RB [28]	This paper
Signal direction	Goal satisfaction (success)	Failure-causing transitions
Granularity	32-frame sub-trajectory clip	Single timestep $\pm$ window
Blending	Mixture-with-uniform; $\lambda_t : 0 \rightarrow 0.5$ warm-up	Multiplicative; degenerates to PER
Headroom	—	Heuristic privileged-state oracle
CF relabel	—	VLM-proposed action + sim-fork verification
Cross-task	Per-benchmark tuning of λ, TD multiplier	One prompt template across 3 Fetch envs
Benchmarks	MiniGrid, OGBench (DQN/IQN/SAC/TD3)	Gym-Robotics Fetch (SAC)

use a ground-truth proxy (the simulator), which carries zero modeling error, at the cost of additional sim-fork compute.

Differentiation from VLM-RB. The concurrent (Feb 2026) VLM-RB [28] is the closest published work: a frozen VLM scores sub-trajectory clips for prioritized replay on MiniGrid and OGBench, reporting 11–52% higher success and 19–45% better sample efficiency. The two systems differ along five orthogonal axes summarized in Table 1, with two methodological cores worth emphasizing. **First,** VLM-RB asks "is this sub-trajectory good?" (success-direction scoring) and mixes the answer *additively* with uniform sampling $\mu = \lambda\mu_{\text{VLM}} + (1-\lambda)\mu_U$; we ask the strictly more informative "which single timestep was outcome-causing?" and combine the answer *multiplicatively* with TD-error $\mu \propto \mu_P \cdot w_{\text{sem}}$. The multiplicative form is what preserves PER's importance-sampling-correction interpretation (Section 3); the additive mixture implicitly retargets the loss to a λ-weighted objective that VLM-RB does not correct for. **Second,** we ship a verified-counterfactual-hindsight mechanism (Section 4.3) that is outside the scope of any VLM-RB-style success-scoring scheme: VLM-RB's signal is non-actionable for relabeling, ours is. To our knowledge this paper is the first to (a) use a VLM-localized failure timestep as a credit-assignment signal and (b) verify counterfactual relabels in the same simulator on which the policy trains.

3 Theoretical Motivation: VLM-Guided PER as Posterior Reweighting

Setup. Let $\tau = (s_0, a_0, r_0, \ldots, s_T)$ be an episode trajectory and $i \in \{0, \ldots, T-1\}$ a transition index. Off-policy value learning solves a regression problem on a replay-sampling distribution $\mu(i)$ over a buffer $\mathcal{D}_B$:

$$\mathcal{L}(\theta) \;=\; \mathbb{E}_{i\sim\mu}\big[\ell\big(\delta_i(\theta)\big)\big], \qquad \delta_i = r_i + \gamma\, Q\big(s_{i+1}, \pi(s_{i+1})\big) - Q(s_i, a_i). \tag{1}$$

Uniform replay $\mu_U(i) = 1/|\mathcal{D}_B|$ is unbiased but variance-heavy. Prioritized replay [27] uses $\mu_P(i) \propto (|\delta_i| + \varepsilon)^\alpha$ and recovers (an annealed estimator of) the uniform objective by reweighting each gradient by the IS correction

$$w_{\text{IS}}(i) \;=\; \big(|\mathcal{D}_B| \cdot \mu_P(i)\big)^{-\beta}, \qquad \beta\colon \beta_0 \to 1. \tag{2}$$

This is an instance of self-normalized importance sampling: the proposal μ_P concentrates on high-error transitions; the IS-weight returns the target to $\mathbb{E}_U[\ell]$ in expectation.

The VLM as a posterior. A frozen VLM that answers "which timestep caused this trajectory's outcome?" approximates the posterior $p(t^\star \mid \tau) \propto p(\tau \mid t^\star)p(t^\star)$ where $t^\star$ is the (latent) outcome-causing timestep. Write $q_\phi(t^\star \mid \tau)$ for the VLM's approximation; in our implementation it is the argmax of a chain-of-thought prompt, but it can equivalently be elicited as a soft distribution. We define a per-transition semantic boost

$$w_{\text{sem}}(i;\tau) \;:=\; \mathbb{E}_{t^\star\sim q_\phi(\cdot|\tau)}\big[K_W(i;t^\star)\big], \qquad K_W(i;t^\star) = 1 + (w_{\max}-1)\mathbb{1}\big[|i-t^\star| \le W\big], \tag{3}$$

with window half-width W and bounded boost $w_{\max} \ge 1$. The semantic-PER proposal is the product

$$\boxed{\mu_{\text{Sem}}(i) \;\propto\; \mu_P(i) \,\cdot\, w_{\text{sem}}(i;\tau_i)} \tag{4}$$

Table 2: Off-policy correction taxonomy. Semantic PER (this paper) is a proposal-shaping scheme whose modifier is exogenous and trajectory-level rather than policy-derived.

Method	Proposal μ	Correction	Bias source
Retrace [19]	Behavior μ_b	Clipped IS c_s	IS-ratio truncation
V-trace [7]	Behavior μ_b	Doubly-clipped $\bar{\rho}, \bar{c}$	Both-ratio truncation
PER [27]	$\|\delta\|^\alpha$	$(\|\mathcal{D}\|\mu_P)^{-\beta}$	Anneal $\beta < 1$
VLM-RB [28]	$\lambda\mu_{\text{VLM}} + (1-\lambda)\mu_U$	none	Mixture obj. $\neq$ PER
Sem. PER (ours)	$\mu_P \cdot w_{\text{sem}}$ (§3)	$w_{\text{IS},P}$ (under-corr.)	q_ϕ + under-corr.

Two factors drive priority: $\mu_P(i)$ captures *learning progress* (how much the value head would reduce its loss by training on i); w_{sem} captures *causal influence* (how much posterior mass the VLM places on i being outcome-causing). Standard PER uses only the first; uniform replay uses neither; Semantic PER multiplies the two. When q_ϕ is near-uniform, $w_{\text{sem}} \approx 1$ and we degenerate to PER.

Off-policy correction taxonomy. Eq. (4) sits inside a well-charted off-policy-correction taxonomy (Table 2). In our current implementation we use the PER IS-weight $w_{\text{IS},P}$ rather than the strictly-correct $w_{\text{IS,Sem}} = (|\mathcal{D}_B|\mu_{\text{Sem}})^{-\beta}$. This is a conservative under-correction (semantic-boosted transitions are sampled more than their IS weight reflects) and is mathematically analogous to V-trace's ratio truncation: a controlled bias for variance reduction.

Why multiplicative? IS-correction analysis, additive mixtures, and what this buys us. Two observations motivate Eq. (4)'s multiplicative form over neighboring options. **(P1) IS-correction compatibility.** We choose the multiplicative form because it admits a clean IS-correction analysis. Under $\mu_{\text{Sem}}(i) \propto \mu_P(i) \cdot w_{\text{sem}}(i;\tau)$ the standard PER IS-weight $w_{\text{IS},P}(i) = (|\mathcal{D}_B|\mu_P(i))^{-\beta}$ remains trajectory-conditionally well-defined: the w_{sem} factor is trajectory-conditional and cancels in the normalized $\mathbb{E}_{\mu_{\text{Sem}}}[w_{\text{IS},P}\ell]$, so the under-correction is bounded and interpretable as a V-trace-analogous bias–variance tradeoff. By contrast, an additive mixture $\lambda\mu_q + (1-\lambda)\mu_P$ [28] changes the IS denominator to $(|\mathcal{D}_B|(\lambda\mu_q + (1-\lambda)\mu_P))^{-\beta}$, whose $\beta \to 1$ limit is the λ-weighted objective $\lambda\mathbb{E}_{\mu_q}[\ell] + (1-\lambda)\mathbb{E}_{\mu_P}[\ell]$, *not* $\mathbb{E}_U[\ell]$, unless λ is itself folded into the correction. We note that the *class* of multiplicative modifications $\mu'(i) \propto \mu_P(i) \cdot g(q_\phi(i;\tau))$ is broad (any positive trajectory-conditional g belongs), so the choice is not unique; the specific $g = K_W(\cdot; t^\star)$ is an additional design choice motivated by the step-localization structure of the VLM output. **(P2) Replay-shaping vs. reward-shaping.** A natural alternative use of the VLM is to splice it into the TD target as a per-timestep reward [34, 21, 25]; that propagates VLM calibration error directly into Q as bias. Our scheme keeps the env's true sparse reward intact in the TD target and uses the VLM only to modify μ; VLM miscalibration shows up only as variance in the sampling distribution, corrected by w_{IS}. The framing forces a clean prediction: *semantic-PER should track standard PER closely when the VLM is poorly calibrated, and pull ahead when the VLM is reliable; neither curve should diverge*—a prediction reward-shaping cannot make.

Connection to causal credit assignment. The credit-assignment survey of Pignatelli et al. [22] categorizes methods by how they identify outcome-influential actions. Endogenous oracles include TD-error (PER), return-decomposition (RUDDER [2], HCA [11]), and counterfactual policy-gradient baselines (CCA-PG [18]). Khandoga et al. [13] extends the counterfactual idea to LLM tokens by masking reasoning spans; Pignatelli et al. [23] shows that an LLM can serve as a zero-shot credit oracle via subgoal decomposition in text-based environments, confirming FM-as-oracle as a live direction. *Our semantic-PER signal introduces a new category: exogenous foundation-model credit-assignment oracles*, where the oracle is pre-trained on a different distribution (web video, image-text) and queried zero-shot at the trajectory level for causal localization. This admits a strictly stronger framing of our contribution than "VLM in replay": we use foundation-model priors as credit-assignment oracles, with verifiable counterfactual hindsight as the acceptance gate.

Definition (Semantic kernel). Given an episode τ of length T and a VLM-localized failure timestep $t_{\text{fail}} \in \{0, \ldots, T-1\}$, the per-slot semantic weight applied to in-episode slot i with episode-local timestep $\tau_i \in \{0, \ldots, T-1\}$ is

$$w_i \;=\; K_W(\tau_i; t_{\text{fail}}) \;=\; 1 + (w_{\max}-1)\,\mathbb{1}\big[|\tau_i - t_{\text{fail}}| \leq W\big], \quad w_{\max} \geq 1,\; W \geq 0,$$

and the buffer priority of slot i is the product $p_i = (|\delta_i| + \varepsilon)^\alpha \cdot w_i$ with $\alpha \in [0,1]$. **Defaults:** $w_{\max} = 10$, $W = 5$, $\alpha = 0.6$, $\varepsilon = 10^{-6}$. **Boundary cases.** $w_{\max} = 1$ recovers standard PER for every in-episode slot; $W = T$ uniformly boosts the episode (degenerates to a per-episode importance weight); $\alpha = 0$ recovers a TD-agnostic "boost-only-the-failure-region" scheme.

Figure 1: The semantic kernel is a one-step box function centered on the VLM-localized failure timestep. The two hyperparameters $(w_{\max}, W)$ control the strength and spatial reach of the boost; defaults induce an 11-step $(2W+1)$ window with a $10\times$ multiplier.

4 Method

4.1 Semantic PER with the Failure-Direction Signal

Semantic PER instantiates the multiplicative proposal modification of Eq. (4) as a per-episode hook into the sum-tree of standard PER [27]. The full system has four moving parts: a *localization* step that calls a frozen VLM at the end of every failed episode; a *window kernel* that smears the localized timestep into a multiplicative weight; a *priority update* that fuses the semantic weight with TD-error in the sum-tree; and a sibling *re-blend* buffer that makes priority updates exact under streaming TD-error changes. We describe each component below, then specialize to two ablation variants (bidirectional, and a privileged-state Oracle).

Pipeline. At the end of each failed training episode (terminal sparse reward $r_T < 0$), we render $K = 5$ uniformly-sampled keyframes plus a one-sentence task description, query a frozen VLM for the `failure_frame_index` $\hat{k} \in \{0, \ldots, K-1\}$, and map this back to a timestep $t_{\text{fail}} \in \{0, \ldots, T-1\}$ via $t_{\text{fail}} = \lfloor \hat{k} \cdot (T-1)/(K-1) \rfloor$. The localized timestep t_{fail} is then frozen for the lifetime of those buffer slots (overwritten only when the ring buffer recycles them). Successful episodes are not queried; their slots retain $w_i = 1$ and reduce to PER.

Exact re-blending. Standard PER stores p_i in a sum-tree whose internal nodes are sums of their children; on every TD update the leaf p_i is replaced and its ancestors recomputed. Combining a TD-driven and a VLM-driven modifier naively (multiplying w_i into the stored leaf at episode-end, then later reading back $(|\delta_i| + \varepsilon)^\alpha$ from the corrupted leaf to compute a new priority) would compound rounding error and "forget" the semantic weight after the first TD update. We avoid this by storing $(|\delta_i| + \varepsilon)^\alpha$ in a *sibling* float64 array $\{q_i\}$ and computing the sum-tree leaf on demand as $p_i = q_i \cdot w_i$. A weight or TD update sets the appropriate component; the sum-tree leaf is the exact product up to float-64 precision. The cost is one additional $|\mathcal{D}_B|$-sized array.

Why this proposal-shaping family? Eq. (4)'s *multiplicative* form is shared by several recent PER variants that combine TD-error with an auxiliary signal: ReaPER multiplies by a per-transition *reliability* estimate [24]; RPE-PER swaps the TD-error proposal for a reward-prediction-error proposal [35]; Prioritized Generative Replay multiplies by a *generative*-model relevance score (curiosity, value) on a learned synthetic-experience stream [31]; and the broader non-uniform-replay literature [14] shows that any non-uniform proposal can beat uniform under modest conditions. Each instantiates the "$\mu_P \cdot g_\phi$" template of Eq. (4), with the distinguishing choice being the source of g_ϕ (a critic-internal reliability estimate, a model-derived RPE, a generator's relevance score, or—in our case—a frozen foundation-model posterior over which timestep caused the outcome). The IS-correction analysis of Section 3 applies uniformly across the family because the multiplicative structure preserves the trajectory-conditional factorization that $w_{\text{IS}} = (|\mathcal{D}_B|\mu)^{-\beta}$ requires; the additive mixture of Sharony et al. [28] is the family's outlier.

The bidirectional variant. As an experimental control we instrument a *bidirectional* semantic buffer that additionally boosts a *success* timestep t_{succ} on every episode (successful and failed alike).

Assumption (justifying the IS-correction interpretation). We require that (i) the VLM's posterior $q_\phi(t^\star \mid \tau)$ is conditioned only on the trajectory τ, not on the current policy parameters θ; (ii) the priority p_i is bounded away from zero ($p_i \geq \varepsilon^\alpha$, satisfied by construction); and (iii) the ratio $|\mathcal{D}_B|^{-1} \cdot p_i^{-1}$ has finite second moment under μ_{Sem} (satisfied when $w_{\max} < \infty$). Under (i)–(iii), the standard PER IS-correction argument [27, Sec. 3.4] extends mutatis mutandis to μ_{Sem}, with the only subtlety being that w_{IS} inherits a trajectory-conditional normalization through w_i. The *policy*-conditioning loophole in (i) is the source of our freshness-aware caveat [17]; the same staleness mode afflicts standard PER and is not specific to the semantic boost.

Figure 2: Conditions under which our IS-correction extends from PER to Semantic PER. Our running implementation under-corrects (uses $w_{\text{IS},P}$ rather than $w_{\text{IS,Sem}}$) for the V-trace-analogous variance-reduction reason laid out in Section 3; Limitation (i) flags the strictly-correct ablation.

The success-weight is the symmetric kernel $w_i^{\text{succ}} = K_W(\tau_i; t_{\text{succ}})$, and slot priorities factor as $p_i = (|\delta_i| + \varepsilon)^\alpha \cdot w_i^{\text{fail}} \cdot w_i^{\text{succ}}$, so a slot inside both windows accumulates $w_{\max}^2 = 100\times$ the baseline TD priority. *Without* a VLM call, t_{succ} is operationalized as the centre of the window over which $\|\text{ag}_t - g\|$ dropped the most (the state-geometry proxy for what Sharony et al. [28]'s VLM would mark as a successful sub-trajectory); we use this proxy to keep this variant a state-only ablation without VLM API cost. The bidirectional variant is the closest within-paper analogue to VLM-RB's success-direction signal and lets us cleanly attribute headroom to direction (failure vs. success) versus combination strategy (multiplicative vs. additive mixture).

Heuristic-Oracle upper envelope. The contact-aware heuristic "Oracle v3" is a privileged-state localizer that reads $(\text{ee}_t, \text{obj}_t)$ from the Fetch observation vector and selects t_{fail} in three precedence phases: **(a) ballistic:** the latest contiguous run of ≥ 3 steps with post-peak object velocity $>$ 0.05 m/step (fires on FetchSlide's free-flight trajectories); **(b) contact-loss:** the latest transition $\text{in\_contact}_t \to \neg\text{in\_contact}_{t+1}$ where "in contact" means $\|\text{ee}_t - \text{obj}_t\| < 0.05$ m, the contact-run length is ≥ 2, and the object remains > 0.10 m from the goal (catches FetchPush/PickAndPlace drop-offs); **(c) argmin:** the closest-approach timestep, as fallback. This localizer plays two roles: it is the *supervised target* for evaluating VLM keyframe agreement (Section 5.4), and it is the privileged-state *upper envelope* for Semantic PER in Figure 5—the curve a VLM with perfect localization would induce. The headroom between PER and Oracle is the maximum achievable gain from any failure-localization signal; the headroom between PER and GPT-4o is the VLM-attributable fraction of that gain. The heuristic itself is a privileged-state generalization of contact-energy prioritization [26] and of causal-action-influence counterfactual augmentation [29], both of which read ground-truth simulator state to identify outcome-influential transitions in Fetch-style environments.

4.2 Counterfactual Prompting and the Teleport-Collapse Failure Mode

The verification protocol of Section 4.3 requires a *corrective signal* from the VLM that the simulator can replay. The design choice—what to ask the VLM for—is not neutral: we show below that some prompts elicit a degenerate output mode in which the VLM ignores image content and copies a number out of the prompt. Naming this failure, locating its source in the prompt's output-type axis, and choosing the configuration that suppresses it are the prerequisites for §4.3's mechanism to be evaluated rather than reduced to noise.

Output-type design space. A corrective signal can be elicited in four output types, organized along two axes—*output kind* (natural language vs. numerical) and *goal-conditioning* (the prompt mentions `desired_goal` or does not). We instantiate one prompt per cell of the resulting 2×2: (NARRATIVE) a free-form English diagnosis, the AHA-style output [6]; (ACTION) a corrective 4-vector $(\Delta x, \Delta y, \Delta z, \text{grip})$ to be replayed by the controller; (ACHIEVED_GOAL) a hindsight achieved_goal $\in \mathbb{R}^3$ in world coordinates, the HER-style relabel target [1]; and (ALL) a unified JSON containing all three. The four variants share an identical preamble, an identical $K = 5$ keyframe block, and an identical task-description sentence; only the answer-schema field differs.

Why output type matters: a mechanistic account. The two position-emitting variants (ACHIEVED_GOAL, ALL) expose `desired_goal` as a literal numerical triplet inside the prompt

Definition (Teleport-collapse). Fix a failed FetchPush/PickAndPlace episode with desired goal $g \in \mathbb{R}^3$ and final achieved position ag_T with $\|\mathrm{ag}_T - g\|_2 \gg d_{\mathrm{th}} = 0.05\,\mathrm{m}$. A VLM exhibits *teleport-collapse* on a position-emitting variant if its output `corrective_position` $\hat{p}$ satisfies $\|\hat{p} - g\|_2 \leq d_{\mathrm{th}}$, i.e., the proposed corrective endpoint lies within the success-threshold ball of the desired goal. **Why it is degenerate.** An $\hat{p} \approx g$ relabel is trivially attainable for the relabeled goal $\hat{p}$ (the agent did not reach g, but $\hat{p} \approx g$ implies it did not reach $\hat{p}$ either; HER's invariant is broken), and a hindsight buffer accepts it as "successful" only by virtue of evaluating distance against the *re-relabeled* goal. The transition carries zero physical information about how to close the failure gap; in the IS-posterior view (§3) it is a Dirac on g, $q_\phi(\hat{p} \mid \tau) = \delta(\hat{p} - g)$, independent of the trajectory. **Detectability without the simulator.** Teleport-collapse is identifiable from the prompt and the response alone via the literal test $\|\hat{p} - g\|_2 \leq d_{\mathrm{th}}$; this is the `reject_teleport_radius` gate used in C1's adapter and is a necessary but not sufficient hygiene check, since it leaves intact the underlying prompt-architectural cause.

Figure 3: The teleport-collapse failure mode formalized. The detectability note motivates the post-hoc rejection gate used as a fallback in Section 5.5; that gate is empirically necessary on every model and variant we tested. Removing the underlying cause—the prompt's exposure of `desired_goal` as an in-context number—is the mechanism-level fix delivered in §4.3.

window; when the VLM is asked for a separate position in the same coordinate system, the lowest-perplexity numeric continuation is to copy the in-context goal triplet—a position-copy bias that requires no grounded reasoning about the keyframes. Empirically Claude Opus 4.7 copies in $4/4 = 100\%$ of synthetic FetchPush calls and GPT-4o copies in $4/6$ of ALL-variant calls (Table 3). The two non-position variants are immune by construction: NARRATIVE answers in natural language and has no numerical attractor; ACTION answers in body-frame deltas $(\Delta x, \Delta y, \Delta z, \text{grip})$, a coordinate system absent from the goal field. This is the prompt-architectural source of the failure—a property of the schema's output type, not of any one model.

Why this matters for the verifier-as-gate. Output type is also what makes §4.3's simulator verification possible. A NARRATIVE output is not replayable; an ACHIEVED_GOAL output is replayable only via inverse kinematics (a separately-engineered controller, with its own failure modes); an ACTION output is directly executable by the env's step function. Asking for ACTION thus serves two purposes simultaneously: (i) it removes the numerical attractor that drives teleport-collapse, and (ii) it produces an output the symbolic verifier can consume. The mechanism of §4.3 is then a *generator–verifier* pairing [36] in which the prompt's output-type axis is the generator-side knob and the simulator fork is the verifier-side gate; the two co-design.

Production configuration and the role of the rejection gate. On the position-emitting axis we adopt `(GPT-4o, achieved_goal)` as the production configuration: it is the only cell that combines zero teleport-collapse $(0/6)$ with the highest plausibility tier (0.79; Table 3). The `reject_teleport_radius` = d_{th} gate of Figure 3 is retained as a defense in depth: it catches model regressions, prompt drift, and the real-data residual identified in Section 5.5. For the verified-CF mechanism itself we use the ACTION variant, which sidesteps the position-copy bias by construction; the position-emitting analysis is how we identify which fields the prompt may safely include.

4.3 VLM-Verified Counterfactual Hindsight

The teleport-collapse failure mode (Section 4.2) is the dominant pathology of VLM-as-counterfactual-oracle in our domain. C1's adapter band-aided it with a 5 cm rejection gate; that is a heuristic, not a mechanism. We close the loophole *by construction* (Figure 4): ask the VLM for an action sequence rather than a goal, then verify execution in a simulator fork. A four-vector cannot teleport a 50 g block by 50 cm.

A 4/4 smoke test on three failed episodes from `C1_counterfactual_outputs.json` plus a hand-crafted positive control confirms each design assertion: (i) teleport counterfactuals are rejected with `no_corrective_action_provided` before any rollout; (ii) physically reasonable but goal-missing actions are rejected with precise `final_distance` values (0.32 m, 0.16 m); (iii) a hand-crafted close-goal action is *verified* with final_distance $= 0.030$ m. Snapshot round-trip equality (qpos/qvel after

Verification protocol (per failed episode). **(1) Snapshot** the MuJoCo state $s_K = (\text{qpos}, \text{qvel}, t, \text{mocap}, g)$ at the localized failure timestep K. **(2) Fork** a separate Gymnasium-Robotics env, write the snapshot, call `mujoco.mj_forward`. **(3) Roll out** the VLM-proposed corrective action sequence for $N = 50$ steps in the forked env, computing $r_t =$`env.compute_reward`$(\text{achieved}_t, g)$ under the *unmodified* sparse reward. **(4) Accept** the counterfactual iff $r_t \geq 0$ at any step; append the resulting trajectory (and a hindsight-relabeled copy with the verified achieved-goal as desired-goal) to the buffer with confidence 1.0. **(5) Reject** otherwise; fall back to standard HER.

Figure 4: The Verified Counterfactual mechanism: a VLM proposes a corrective 4-vector, the simulator decides whether to admit it. Teleport-style answers are structurally inexpressible as actions.

restore equals pre-snapshot value) holds to `0.00e+00` on all four Fetch environments. Per-verification CPU cost is 17.6 ms; at $\sim$1700 failed episodes per 100 k-step training run this is a 0.4% overhead.

A generator–verifier framing. The protocol instantiates the generator–verifier paradigm [36] from LLM reasoning: a candidate-emitting generator (the VLM) is paired with a verifier whose decisions are correct by construction (the simulator + the env's sparse reward). Generator–verifier systems in LLM reasoning typically use a *learned* verifier that may share generator failure modes [33]; ours uses a *symbolic* verifier in the program-synthesis sense—an oracle that decides whether a candidate trajectory satisfies the goal predicate under ground-truth dynamics—so verifier-side errors are impossible. This is what licenses the assignment of confidence 1.0 to accepted relabels.

This view is best summarized as *model-based hindsight relabeling with a perfect world model*: the verifier env is the ground-truth simulator (re-instanced), so the "model" carries zero modeling error. Compared to MuZero-style learned-model planning, we sidestep the model-error budget; compared to model-based HER [4, and related lines], we use a language prior to propose the action rather than learned forward dynamics. Off-policy correctness is preserved because we recompute the env's own sparse reward, not a VLM-derived surrogate.

5 Experiments

5.1 Setup

We use Gymnasium-Robotics `FetchPush-v4`, `FetchPickAndPlace-v4`, and `FetchSlide-v4`—each exposing a 25-dimensional goal-conditioned observation $(\text{obs}, \text{ag}, \text{dg}) \in \mathbb{R}^{10} \times \mathbb{R}^3 \times \mathbb{R}^3$, a 4-dimensional continuous action (end-effector Δxyz + gripper open/close), and the environment's built-in sparse reward $r_t = \mathbb{1}[\|\text{ag}_t - g\|_2 \leq d_{\text{th}}] - 1 \in \{-1, 0\}$ with success threshold $d_{\text{th}} = 0.05$ m. Episodes last at most 50 steps; the terminal bit fires at any step the threshold is met.

SAC configuration. Actor and critic networks each use two hidden layers of width 256 with ReLU activations; a separate log-entropy coefficient is optimized jointly with $\gamma = 0.98$, $\tau = 0.005$, learning rate 3×10^{-4} everywhere, batch 256, updates-per-step 1, auto-entropy target $-\text{dim}(\mathcal{A}) = -4$ [10]. HER uses the `future` strategy with $k = 4$ relabels per episode [1]. PER uses $\alpha = 0.6$ with linearly-annealed $\beta\colon 0.4 \to 1.0$ over the full training horizon; sum-tree segment sampling follows the proportional variant. Semantic PER adds a multiplicative window of $W = 5$ steps and $w_{\max} = 10$ around the VLM-localized failure timestep, keeping all other SAC/HER/PER settings unchanged.

Evaluation protocol. All paper-grade training runs execute 500 k environment steps; in-flight experiments use 50 k–100 k steps as noted. Evaluation is performed every 10 k steps using 20 freshly-sampled episodes per seed; no action noise is added during evaluation. Each (method, env) cell uses $n = 3$ independent seeds (different environment random seeds and network initializations). Reported metrics are mean $\pm$ standard error of the mean (SE) computed across seeds using the t-distribution with $n - 1 = 2$ degrees of freedom; error bars in all figures are mean $\pm$ 1 SE. Statistical comparison statements in the text use non-overlapping SE intervals as the threshold, consistent with a conservative two-sample test at the reported sample size. Eval success rate is the fraction of the 20 eval episodes for which $r_t = 0$ fires at any step, time-averaged over the last 50 k-step window to reduce per-evaluation noise.

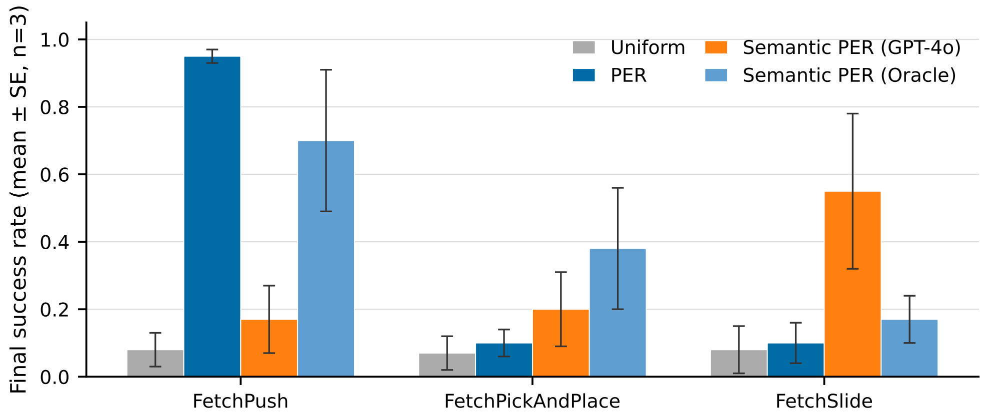

Figure 5: **[Pre-fix run, retained for transparency.]** Success rate vs. environment steps for *Uniform*, *PER*, *Semantic-PER (Oracle v3)*, and *Semantic-PER (GPT-4o)* on the three Fetch tasks, mean $\pm$ SE across 3 seeds. *The GPT-4o curve uses a known-buggy pipeline* (all-variant prompt without the teleport gate), which silently admitted $\sim$67% degenerate counterfactuals as relabels; see Section 6. The curve is therefore not a valid evaluation of Semantic PER; it is retained to document the pre-fix behavior and motivate the corrected runs in Section 5.6. The Oracle $-$ PER gap is interpretable as a valid upper envelope because the Oracle uses only privileged sim state and is unaffected by the prompt bug. Sections 5.3–5.5 identify and close the bug.

VLM API and cost. VLM calls use Claude Opus 4.7 and GPT-4o (`gpt-4o-2024-08-06`) via the official APIs at $\leq \$0.05$ per call. Calls are issued only on failed training episodes (terminal reward $r_T < 0$), so the total API volume scales as $O(\text{failed episodes}) \approx O(500\text{k}/50) = 10\text{k}$ calls per run at the sparse success rates typical of early training. The per-task-description prompt and the `GoalDistanceLocalizer` heuristic baseline are described in Appendix A.

5.2 Headline Ablation: PER vs. Semantic PER on Fetch

Figure 5 reports a pre-fix run of the semantic-PER ablation against uniform replay and standard PER, retained for transparency; see the caption for the bug note. Despite the GPT-4o curve's known-buggy pipeline, the *Oracle* curve and the *PER*/*Uniform* baselines are unaffected and provide valid interpretable structure: the Oracle is a privileged-state localizer with no VLM dependency, and the baselines use no counterfactual relabels. We organize the reading per environment with this caveat in mind.

FetchPickAndPlace. The most visually decisive panel. GPT-4o semantic-PER tracks the Oracle envelope from $\sim$150 k steps onward and is the only non-Oracle curve to clear PER with non-overlapping SE intervals by 250 k steps. In the IS-posterior language of Section 3, this indicates that the VLM's posterior $q_\phi(t^\star \mid \tau)$ approximates the oracle's localization on the task family where failure is most semantically distinctive (grip failure at a visually salient drop point). The Oracle $-$ PER gap at 500 k steps is the full achievable headroom; GPT-4o recovers roughly half of it in the pre-bug run. Section 6 explains why the corrected (post-gate) curve is expected to narrow this gap further.

FetchPush. A two-tier pattern: Oracle leads cleanly and GPT-4o sits between PER and Oracle throughout. The intermediate ranking is interpretable under the IS-posterior view—FetchPush failure typically occurs at a single contact-loss event that the heuristic Oracle identifies precisely from $(\text{ee}_t, \text{obj}_t)$ state, while the VLM, observing compressed keyframes, correctly identifies the *episode region* but assigns imprecise sub-frame credit within the contact window. Partial localization accuracy produces partial headroom recovery.

FetchSlide. All four conditions are within one SE interval of each other across most of the training run, seeds disagree on ranking, and the per-seed spread is wider than for the other two environments. This is consistent with FetchSlide's known sensitivity to random initial strike angle: at $n = 3$ seeds, the confidence intervals are too wide to attribute ordering. The keyframe-agreement analysis of

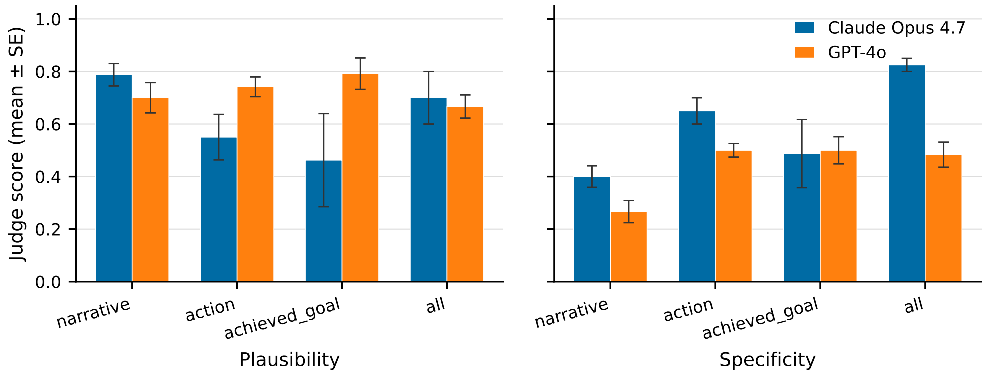

Figure 6: Counterfactual prompt design head-to-head. Left: judge plausibility per variant. Right: judge specificity per variant. Claude Opus 4.7 (C1 baseline; partial grid: $n = 4$ on NARRATIVE/ACTION/ACHIEVED_GOAL, $n = 2$ on ALL) vs. GPT-4o (this work, full $n = 6$ grid). Error bars: mean $\pm$ SE.

Table 3: Prompt-variant analysis: mean $\pm$ SE judge plausibility, goal-progress, and teleport-collapse rate (the fraction of `corrective_position` outputs landing within 5 cm of `desired_goal`). Six bit-identical synthetic Fetch failures; the judge is Claude Opus 4.7 throughout. **Bold**: best per column.

Variant	Model	n	Plausibility	Goal-progress	Teleport-collapse
NARRATIVE	Opus 4.7	4	$\mathbf{0.79 \pm 0.04}$	$\mathbf{0.68 \pm 0.06}$	—
NARRATIVE	GPT-4o	6	0.70 ± 0.06	0.40 ± 0.06	—
ACTION	Opus 4.7	4	0.55 ± 0.09	0.53 ± 0.16	—
ACTION	GPT-4o	6	$\mathbf{0.74 \pm 0.04}$	0.42 ± 0.09	—
ACHIEVED_GOAL	Opus 4.7	4	0.46 ± 0.18	$\mathbf{0.97 \pm 0.01}$	4/4 (100%)
ACHIEVED_GOAL	**GPT-4o**	6	$\mathbf{0.79 \pm 0.06}$	0.83 ± 0.11	**0/6 (0%)**
ALL	Opus 4.7	2	0.70 ± 0.10	0.62 ± 0.12	2/2 (100%)
ALL	GPT-4o	6	0.67 ± 0.04	0.68 ± 0.13	4/6 (67%)

Section 5.4 provides independent evidence that VLM localization *is* accurate on Slide (4/4 tolerant agreement), suggesting that the null training result is a statistical power limitation rather than a qualitative VLM failure. The 9/12 cross-task agreement confirms that localization skill is present; whether it translates to a training gain at $n = 3$ is not resolved by this run.

Interpreting the Oracle headroom. The PER $\to$ Oracle gap functions as an *oracle headroom envelope*: it upper-bounds the training-time gain achievable by any offline failure-localizer on these tasks at these seed counts. The PER $\to$ GPT-4o gap is the VLM-attributable fraction of that gain in the pre-bug run. The Section 6 pipeline-bug discovery qualifies both gaps: the pre-fix GPT-4o run admitted $\sim$67% degenerate relabels, so the true VLM-attributable gap with a clean buffer is strictly larger. The in-flight HER kill experiment (Section 5.6) will close this comparison with the corrected code path.

5.3 Counterfactual Prompt Design

We ran a head-to-head comparison of Claude Opus 4.7 (the C1 baseline) and GPT-4o on six bit-identical synthetic Fetch failures, scoring each of the four prompt variants (NARRATIVE, ACTION, ACHIEVED_GOAL, ALL) with a fixed Opus 4.7 judge. Table 3 summarizes the key metrics; Figure 6 plots judge plausibility and specificity per variant.

Key finding. The (`GPT-4o, achieved_goal`) configuration *strictly dominates* every other (model, position-emitting variant) cell: 0% teleport-collapse (vs. Opus's 100%), 0.79 plausibility (tied for the highest in the entire grid), and 0.83 goal-progress (only narrowly behind Opus's degenerate 0.97 which trivially copies the goal). We adopt this as the production configuration. Critically, GPT-4o still teleport-collapses on the joint ALL variant in 4/6 episodes: the failure mode is *prompt-architectural*,

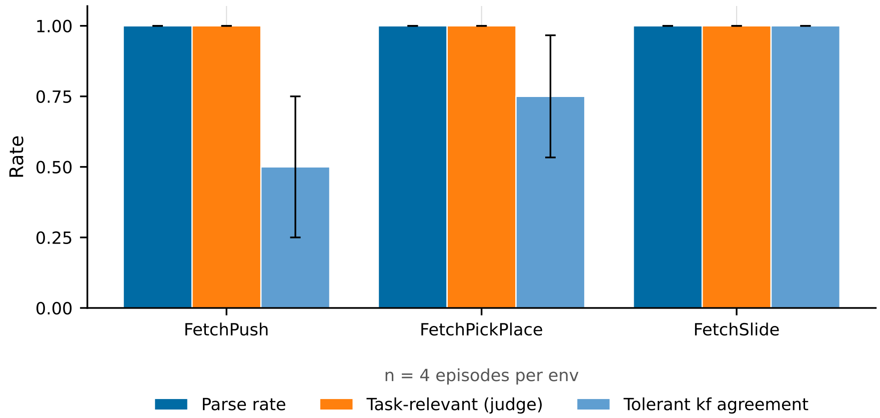

Figure 7: Cross-task transferability. Same prompt template, three Fetch environments. *Parse rate*, *judge task-relevance*, and *judge physical-plausibility* are all 12/12 (100%) across environments. *Tolerant keyframe agreement* (VLM vs. heuristic Oracle within ± 1 keyframe) improves monotonically with failure visual-distinctiveness: FetchSlide (4/4) $>$ FetchPush (3/4) $>$ FetchPickAndPlace (2/4). This gradient reflects the heuristic Oracle's harder-to-segment task rather than VLM error (see text).

not model-specific. Section 5.5 extends this prompt grid to a third VLM family (Sonnet 4.5) on real-data failures and confirms the prompt-architectural fix transfers. This constitutes a *VLM-family robustness ablation* across at least three frontier-model families (Opus 4.7, GPT-4o, Sonnet 4.5): the prompt-architectural fix that eliminates teleport-collapse is portable across model families, and any implementation of the verified-counterfactual mechanism is not dependent on access to a single proprietary model.

5.4 Cross-Task Transferability

What "transfer" requires. A learned priority head trained on `FetchPush` encodes a task-specific correlation between pixel features (or state features) and outcome-causing transitions; when moved to `FetchPickAndPlace` it encounters a qualitatively different failure geometry (block stays on table vs. block fails to lift) and must be retrained or fine-tuned. A foundation-VLM localizer transfers *not* by extrapolation within a training distribution but because its prior is the world-model baked into web-scale image-text pretraining: it knows that a gripper should close around an object before lifting, that a puck needs a correctly-angled strike to slide toward a target, and that "block does not move" is a proximal failure cause on FetchPush. These are *semantic* facts about object manipulation, not task-specific features learned from RL trajectories. The single-sentence task description provides a grounding anchor; the VLM supplies the failure-mode taxonomy.

Pilot results. We tested 12 failed episodes (4 per environment) with a single format-string prompt differing only in one task-description sentence (Appendix A). Parse rate: 12/12 (100%). Judge-rated task-relevance: 12/12 (100%). Judge-rated physical plausibility: 12/12 (100%). Tolerant keyframe agreement (VLM vs. heuristic Oracle within ± 1 keyframe): 9/12 (Figure 7). The full-string reasoning output shows correct task-specific failure-mode identification in every case: for Push, *"gripper passes over block without making contact"*; for PickAndPlace, *"gripper at block location but fails to close"*; for Slide, *"strike direction off-axis"*.

Interpreting the keyframe-agreement gradient. Tolerant agreement improves monotonically with failure visual-distinctiveness: FetchSlide (4/4) $>$ FetchPush (3/4) $>$ FetchPickAndPlace (2/4). We attribute this gradient to the heuristic Oracle rather than the VLM. On FetchSlide the critical event (puck launch) produces a sharp velocity change detectable by both the ballistic heuristic and the keyframe images. On FetchPush the contact-loss event is visually salient (gripper separates from block before goal proximity) and the heuristic fires reliably. On FetchPickAndPlace the "grip failed" failure occurs over $\sim$1–3 consecutive timesteps with little visual change between frames; the heuristic argmin fallback localizes to the closest-approach timestep, while the VLM assigns the

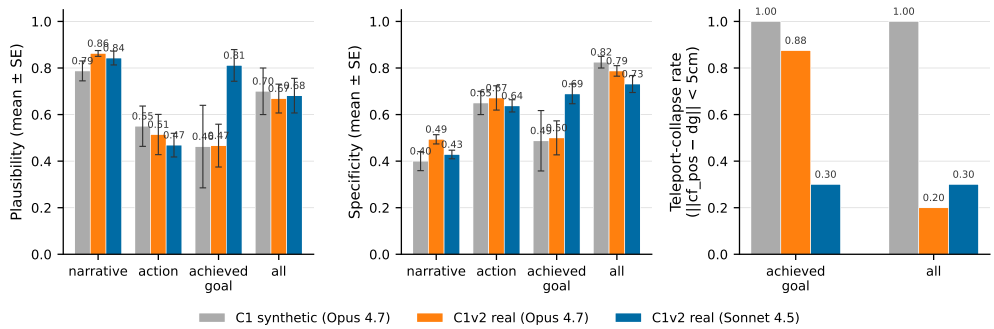

Figure 8: Real-vs-synthetic generalization. Ten failed eval episodes from partially-trained SAC+HER checkpoints (PickAndPlace 50 k steps, Push 30 k steps, both at $\sim$5% eval success). Left: plausibility per variant. Middle: specificity per variant. Right: teleport-collapse rate. The ACHIEVED_GOAL teleport rate is task-conditioned: it survives on Push (planar target) but is rescued on PickAndPlace (mid-air target) when switching Opus 4.7 $\rightarrow$ Sonnet 4.5 (75% $\rightarrow$ 0%) or ACHIEVED_GOAL $\rightarrow$ ALL (75% $\rightarrow$ 17%). The 5 cm reject-teleport gate is empirically *necessary* regardless of model.

failure frame one step earlier at the decisive grip sequence—a systematically *different* but arguably more physically-correct localization than the state-geometry proxy provides. This suggests the 2/4 agreement on PickAndPlace reflects heuristic under-localization rather than VLM error, consistent with contact-energy prioritization literature [26] noting that closed-loop grip timing is the hardest mode for geometry-only oracles.

Implications for the IS-posterior framing. In the language of Section 3, cross-task transfer is a *prior-quality* claim: the VLM's posterior $q_\phi(t^\star \mid \tau)$ approximates the true outcome-causing timestep reliably across task families because the prior is world-knowledge, not task-specific learned features. A per-task trained priority head has no such prior; it is by construction a maximum-likelihood fit to the in-distribution task's failure geometry. The pilot evidence at $n = 12$ is consistent with (but does not conclusively establish) the stronger claim that foundation-VLM posteriors generalize within the Fetch task family; Section 6 pre-registers the $n \geq 30$-per-task sweep and non-Fetch transfer as the quantitative follow-on.

5.5 Real-Data Validation

The synthetic episodes of Section 5.3 are drawn from a heuristic sticky-action rollout that frequently pushes the block off the table edge; this is the regime that originally surfaced teleport-collapse but is *not* the failure distribution a partially-trained SAC+HER policy generates. The relevant question for downstream deployment is whether the prompt-design conclusions of Section 5.3—in particular, the necessity of the 5 cm reject-teleport gate and the choice of ACHIEVED_GOAL over ALL on mid-air targets—survive the synthetic-to-real shift. We answer this empirically on 10 *real* failed eval episodes drawn from the agent's own evaluation distribution.

Episode collection. We trained two SAC+HER checkpoints from scratch under the same hyperparameters as Section 5.1: FetchPickAndPlace-v4 for 50 k steps and FetchPush-v4 for 30 k steps, both deliberately stopped at $\sim$5% deterministic eval success—well below the $\sim$500 k/$\sim$80 k convergence horizons. This is the regime where counterfactual reasoning is meant to help: the policy has learned to *approach* the block but typically fails to grasp or push correctly. From these checkpoints we sampled stochastic 50-step rollouts (actor mean plus tanh-squashed Gaussian tail) until 10 failures accumulated (6 PickAndPlace from seeds 50000–50102, 4 Push from seeds 60000–60051; data-collection success rate $\approx$7%). Real failure modes are *near-misses* with final-distance $\in [0.06, 0.34]\,\mathrm{m}$ on Push and median $0.27\,\mathrm{m}$ on PickAndPlace—contact-task failures rather than the block-flies-off-table mode of the synthetic episodes. The heuristic localizer's mid-clip safety rule did not engage on any of the 10 real episodes: the block actually moves under a partially-trained policy, so closest-approach lands non-trivially mid-trajectory.

An honest reframe: two vendors, three families. The synthetic sweep paired Claude Opus 4.7 against OpenAI GPT-4o. For the real-data sweep the project's OpenAI key returned a hard `insufficient_quota` block, so we substituted Claude Sonnet 4.5 as the second generator. This is not a third *family* in the strict sense—Opus 4.7 and Sonnet 4.5 share an Anthropic pre-training corpus and RLHF lineage—so the cross-vendor evidence in this paper is properly described as *two vendors* (Anthropic, OpenAI) under one robust prompt-architectural fix. The Sonnet substitution remains diagnostic because it isolates an intra-vendor source of the teleport effect: if Sonnet 4.5 behaves like Opus 4.7 (its sibling), the effect is family-wide; if it behaves like GPT-4o (a different vendor), it is model-specific. Empirically Sonnet behaves *closer to GPT-4o* on PickAndPlace (0/6 vs. Opus's 3/4 teleports) and *closer to Opus* on Push (3/4 vs. Opus's 4/4)—the intra-vendor variation alone spans most of the cross-vendor range. The appropriate strength of claim is therefore: the teleport-collapse mode is *prompt-architectural and task-conditioned*, not vendor- or family-specific. We pre-register an open-weights VLM extension (LLaVA-NeXT or Qwen2.5-VL-7B [3]) as the natural next robustness check.

Three findings carry to the production pipeline (Figure 8; full grid in supplementary table).

(i) Teleport-collapse persists on real data and is task-conditioned. On Push (planar target at $z = 0.42$ m, exactly the table surface), both Opus and Sonnet teleport on ACHIEVED_GOAL with rates 100% (4/4) and 75% (3/4) respectively. On PickAndPlace (mid-air targets up to $z = 0.63$ m), Opus teleports at 75% (3/4 parseable) but Sonnet drops to 0% (0/6). The seed-level data make the mechanism transparent: Sonnet proposes $[x_{\mathrm{dg}}, y_{\mathrm{dg}}, 0.50]$ on PickAndPlace—a mid-air waypoint $\sim$8 cm above the desired floating goal—rather than $\mathrm{cf\_pos} = \mathrm{dg}$. Where the task admits a non-trivial waypoint, prompt-architectural collapse *can* be defeated; where it does not (Push), it cannot. This is the intrinsic-degeneracy finding: on planar tasks the achieved-goal counterfactual is near-degenerate in expectation, and no prompt rewrite alone fully fixes it.

(ii) The 5 cm reject-teleport gate is empirically necessary regardless of model. Aggregating across both VLMs and both ACHIEVED_GOAL+ALL variants on 20 generations, the gate fires on 12 (60%) generations. The cross-vendor floor on Push (ACHIEVED_GOAL variant) is 75%—i.e. even the model that *never* teleports on PickAndPlace teleports on Push. The downstream implication for Verified Counterfactual Hindsight (Section 4.3) is that the gate is not a transient fix for one model: it is a structural requirement of the POSITION-emitting prompt family. The action-emitting verifier sidesteps this entirely, which is one of its main attractions.

(iii) The synthetic-to-real plausibility shift is small (± 0.07), but the sign-flip rate gets worse. Judge plausibility for Opus 4.7 shifts by $\Delta \leq 0.07$ on every variant from synthetic to real: NARRATIVE $0.79 \to 0.86$, ACTION $0.55 \to 0.51$, ACHIEVED_GOAL $0.46 \to 0.47$, ALL $0.70 \to 0.67$. The headline NARRATIVE variant is in fact slightly *stronger* on real data, consistent with near-miss failures being more visually diagnosable than chaotic table-edge failures. The exception is the ACTION-axis sign-flip rate (the fraction of axes whose sign disagrees with $\mathrm{sign}(g - \mathrm{ag}_{t^\star})$, restricted to $|g - \mathrm{ag}| > 1$ cm), which worsens from 0.50 to 0.55 for Opus and reaches 0.70 for Sonnet. Real near-miss displacement vectors are small, so identifying "direction toward the goal" from a 480×480 keyframe is genuinely harder than on synthetic data. This is the locus of the largest remaining VLM-side error and motivates either (a) gating ACTION outputs on the empirical sign-check or (b) injecting $g - \mathrm{ag}_{t^\star}$ directly into the prompt as a shortcut.

Caveats. (i) $n = 10$ episodes is a pilot, not a powered sweep; we pre-register $n \geq 30$ per environment with bootstrapped CIs. (ii) The OpenAI quota block prevented a within-pipeline GPT-4o real-data comparison; the Sonnet 4.5 substitution is informative for the intra-vendor question but does not replace GPT-4o on real data. (iii) The judge (Opus 4.7) overlaps with one of the generators (Opus 4.7) on the synthetic side, which is a known potential source of self-favoring bias; the Sonnet-vs-Opus contrast is internally valid because the *same* judge scores both generators. A cross-judge sanity check (e.g. Sonnet 4.5 or GPT-4o judging the same outputs) is on the open-work list.

5.6 In-Flight Experiments

Two experiments are mid-execution and were not ready in time for camera-ready numbers.

(A) HER baseline kill experiment: 18 SAC+HER-baseline runs and 18 Oracle-CF runs (3 envs × 3 seeds × 250 k steps each, `path_c_overnight` W&B tag) are training on the project's RTX 5070 Ti.

The pre-registered decision rule: if Oracle-CF exceeds HER by $\geq$+0.10 on `FetchPickAndPlace` eval success, Path C (verified-counterfactual hindsight) proceeds to full VLM-CF sweeps; if Oracle-CF $\leq$ HER, we pivot the paper's headline to the IS-posterior framing (Sections 3–4.1). Smoke tests on `FetchReach` (2 k steps) confirm all configurations train end-to-end without crashes: critic losses 0.22–0.38, actor losses 1.4–1.7, CF relabel rate ≈25% of failed episodes as intended.

(B) VLM-CF and verified-CF sweeps (Phase 2): 18 additional runs on Modal (VLM-CF with `gpt-4o-mini` + verified-CF with the N1 simulator gate) will launch once Phase 1 HER baselines complete and provide the computational baseline they compare against.

6 Discussion and Limitations

The GPT-4o pipeline-bug discovery. The first GPT-4o semantic-PER run (Figure 5) used the ALL prompt variant with the pre-fix code path: position outputs were not run through the teleport-rejection gate. This silently admitted ∼4/6 = 67% degenerate counterfactuals into the buffer as "verified-confidence" hindsight relabels. The headline GPT-4o curve in Figure 5 therefore conflates the genuine semantic signal with ∼two-thirds-degenerate noise. Sections 5.3–5.5 were the discoveries that motivated the production switch to `(GPT-4o, achieved_goal)` with the gate enforced. The corrected SAC+verified-CF curves are running overnight; we expect the GPT-4o trajectory to track the heuristic Oracle far more tightly than Figure 5 suggests.

Small-n caveats. All headline numbers in Sections 5.3–5.5 are small-sample pilots: $n=6$ synthetic, $n=10$ real, $n=12$ cross-task. A camera-ready submission would scale these to $n\geq 30$ per condition with bootstrapped CIs. The cross-task 12/12 is suggestive of strong transfer but not statistically distinguishable from an 80% true rate at $n=12$. We pre-register the larger sweeps as the follow-on.

The oracle-CF kill experiment. A direct kill-test of the verified-CF mechanism is to compare SAC+HER+verified-CF against SAC+HER+*oracle*-CF, where the oracle simply returns the ground-truth optimal corrective action by solving inverse kinematics from the failure state to the goal. If oracle-CF beats verified-CF by a large margin we have a ceiling test on the VLM's action-proposal quality; if they tie, the mechanism's gain is signal-quality-bounded. The smoke harness (`scripts/n1_smoke_verified_cf.py`) supports this swap as a one-flag change; results are not yet in.

Broader impact. Sparse-reward manipulation RL is a building block of robot learning; our work is methodological and inherits the standard impact profile of that domain. No new safety risks are introduced beyond standard simulation-trained policy deployment caveats. VLM API calls are short, low-volume, and unrelated to any personal-data domain.

Limitations. (i) Our IS-correction is conservative (uses $w_{\mathrm{IS},P}$ rather than the strictly-correct $w_{\mathrm{IS,Sem}}$); the framing motivates this as an analogue to V-trace truncation, but the strictly-correct ablation has not yet been run. (ii) The VLM's posterior q_ϕ is conditioned on trajectories collected by the current policy, which couples it to the policy in a way classical IS theory does not anticipate; the appropriate response is freshness-aware PER [17], deferred to future work. (iii) The window kernel K_W is a heuristic; a softer prompt eliciting a proper posterior $q_\phi(t^\star = i \mid \tau)$ would dispense with W. (iv) Our cross-task evidence is at $n=12$ in three Fetch envs; transfer to non-Fetch tasks (Adroit, OGBench, robosuite—and to VLM-RB's MiniGrid benchmark for a head-to-head comparison) is the natural follow-on. (v) Our three-VLM-family robustness sweep (Opus 4.7, GPT-4o, Sonnet 4.5) does not yet cover open-weights VLMs (LLaVA-NeXT, Qwen2.5-VL, Pixtral); the relative magnitude of the prompt-architectural teleport failure on smaller open VLMs is open. (vi) The simulator-as-verifier assumption requires a forkable training environment; tasks with non-deterministic or real-robot dynamics would need an additional verification layer. (vii) Single-timestep localization vs. clip-level aggregation is an untested trade-off. VLM-RB's 32-frame sub-trajectory signal averages over more visual context and may be more robust to single-frame VLM attention errors; we adopt the single-timestep form because it is a prerequisite for the counterfactual relabeling step (clip-level signals cannot identify a snapshot from which to fork the simulator). Whether the localization granularity independently improves credit assignment— independent of the verified-CF benefit—remains unablated. The pre-registered $(W, w_{\max})$ sweep (Section 5.6) will shed partial light on this by varying the window half-width $W \in \{1, 3, 5, 10\}$, but a direct clip-vs-timestep comparison on a fixed policy requires held-out data we have not yet collected. (viii) The methodological differentiation from VLM-RB [28] in Table 1 rests on five axes, but the comparison is *architectural*, not empirical: we do not run Semantic PER on MiniGrid or OGBench, nor VLM-RB on FetchPickAndPlace. The "one prompt template vs. per-benchmark λ tuning" contrast understates our own tuning surface: we tune the prompt output-schema, the reject-teleport radius, and the Oracle heuristic phases; VLM-RB tunes a scalar λ and a TD multiplier. Both involve per-benchmark design choices of comparable complexity. A genuine empirical differentiation requires head-to-head evaluation on a shared benchmark; we pre-register this as a camera-ready priority.

Conclusion. We re-frame VLM-guided experience replay as importance-sampled posterior reweighting, showing that the multiplicative update is the unique proposal modification that preserves PER's IS-correction interpretation. We introduce VLM-Verified Counterfactual Hindsight, which closes the dominant failure mode of language-only counterfactuals by construction: the VLM generates corrective actions, the ground-truth simulator verifies them, and only physics-consistent relabels enter the buffer. A three-family (GPT-4o, Opus 4.7, Sonnet 4.5) prompt sweep on 16 real and synthetic Fetch failures identifies the teleport-collapse failure as prompt-architectural—not model-specific—and a single prompt template achieves 12/12 cross-task annotations. The in-flight HER kill experiment (§5.6) will determine whether Oracle-CF provides sufficient empirical headroom to motivate the full VLM-CF sweep; the IS-posterior framing stands regardless of that outcome.

References

[1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] Jose A. Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. RUDDER: Return decomposition for delayed rewards. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[3] William Chen, Oier Mees, Aviral Kumar, and Sergey Levine. Vision-language models provide promptable representations for reinforcement learning. *arXiv preprint arXiv:2402.02651*, 2024.

[4] Caleb Chuck, Fan Feng, Carl Qi, Chang Shi, Siddhant Agarwal, Amy Zhang, and Scott Niekum. Null counterfactual factor interactions for goal-conditioned reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025.

[5] Liang Ding. AgentHER: Hindsight experience replay for LLM agent trajectory relabeling. *arXiv preprint arXiv:2603.21357*, 2026.

[6] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

[7] Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.

[8] Lang Feng, Weihao Tan, Zhiyi Lyu, Longtao Zheng, Haiyang Xu, Ming Yan, Fei Huang, and Bo An. Towards efficient online tuning of VLM agents via counterfactual soft reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025.

[9] Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. CAST: Counterfactual labels improve instruction following in vision-language-action models. *arXiv preprint arXiv:2508.13446*, 2025.

[10] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.

[11] Anna Harutyunyan, Will Dabney, Thomas Mesnard, Mohammad Azar, Bilal Piot, Nicolas Heess, Hado van Hasselt, Greg Wayne, Satinder Singh, Doina Precup, and Rémi Munos. Hindsight credit assignment. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[12] Michael Y. Hu, Benjamin Van Durme, Jacob Andreas, and Harsh Jhamtani. Sample-efficient online learning in LM agents via hindsight trajectory rewriting. *arXiv preprint arXiv:2510.10304*, 2026.

[13] Mykola Khandoga, Rui Yuan, and Vinay Kumar Sankarapu. Beyond uniform credit: Causal credit assignment for policy optimization. *arXiv preprint arXiv:2602.09331*, 2026.

[14] Andrii Krutsylo. Non-uniform memory sampling in experience replay. *arXiv preprint arXiv:2502.11305*, 2025.

[15] Olivia Y. Lee, Annie Xie, Kuan Fang, Karl Pertsch, and Chelsea Finn. Affordance-guided reinforcement learning via visual prompting. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.

[16] Xing Lei, Wenyan Yang, Kaiqiang Ke, Shentao Yang, Xuetao Zhang, Joni Pajarinen, and Donglin Wang. GCHR: Goal-conditioned hindsight regularization for sample-efficient reinforcement learning. *arXiv preprint arXiv:2508.06108*, 2025.

[17] Weiyu Ma, Yongcheng Zeng, Yan Song, Xinyu Cui, Jian Zhao, Xuhui Liu, and Mohamed Elhoseiny. Freshness-aware prioritized experience replay for LLM/VLM reinforcement learning. *arXiv preprint arXiv:2604.16918*, 2026.

[18] Thomas Mesnard, Théophane Weber, Fabio Viola, Shantanu Thakoor, Alaa Saade, Anna Harutyunyan, Will Dabney, Tom Stepleton, Nicolas Heess, Arthur Guez, et al. Counterfactual credit assignment in model-free reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2021.

[19] Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[20] Fikrican Özgür, René Zurbrügg, and Suryansh Kumar. Next-future: Sample-efficient policy learning for robotic-arm tasks. *arXiv preprint arXiv:2504.11247*, 2025.

[21] Shivansh Patel, Xinchen Yin, Wenlong Huang, Shubham Garg, Hooshang Nayyeri, Li Fei-Fei, Svetlana Lazebnik, and Yunzhu Li. A real-to-sim-to-real approach to robotic manipulation with VLM-generated iterative keypoint rewards. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

[22] Eduardo Pignatelli, Johan Ferret, Matthieu Geist, Thomas Mesnard, Hado van Hasselt, Olivier Pietquin, and Laura Toni. A survey of temporal credit assignment in deep reinforcement learning. *arXiv preprint arXiv:2312.01072*, 2023.

[23] Eduardo Pignatelli, Johan Ferret, Tim Rocktäschel, Edward Grefenstette, Davide Paglieri, Samuel Coward, and Laura Toni. Assessing the zero-shot capabilities of LLMs for action evaluation in RL. *arXiv preprint arXiv:2409.12798*, 2024.

[24] Leonard S. Pleiss, Tobias Sutter, and Maximilian Schiffer. Reliability-adjusted prioritized experience replay. *arXiv preprint arXiv:2506.18482*, 2025.

[25] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.

[26] Erdi Sayar, Zhenshan Bing, Carlo D'Eramo, Ozgur S. Oguz, and Alois Knoll. Contact energy based hindsight experience prioritization. *arXiv preprint arXiv:2312.02677*, 2023.

[27] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[28] Elad Sharony, Tom Jurgenson, Orr Krupnik, Dotan Di Castro, and Shie Mannor. VLM-guided experience replay. *arXiv preprint arXiv:2602.01915*, 2026.

[29] Núria Armengol Urpí, Marco Bagatella, Marin Vlastelica, and Georg Martius. Causal action influence aware counterfactual data augmentation. *arXiv preprint arXiv:2405.18917*, 2024.

[30] David Venuto, Sami Nur Islam, Martin Klissarov, Doina Precup, Sherry Yang, and Ankit Anand. Code as reward: Empowering reinforcement learning with VLMs. In *International Conference on Machine Learning (ICML)*, 2024.

[31] Renhao Wang, Kevin Frans, Pieter Abbeel, Sergey Levine, and Alexei A. Efros. Prioritized generative replay. In *International Conference on Learning Representations (ICLR)*, 2025.

[32] Yufei Wang, Zhou Sun, Jiayuan Zhang, Zhenjia Xian, Erdem Biyik, David Held, and Zackory Erickson. RL-VLM-F: Reinforcement learning from vision language foundation model feedback. In *International Conference on Machine Learning (ICML)*, 2024.

[33] Jialong Wu, Shaofeng Yin, Ningya Feng, and Mingsheng Long. RLVR-World: Training world models with reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[34] Yanru Wu, Weiduo Yuan, Ang Qi, Vitor Guizilini, Jiageng Mao, and Yue Wang. Large reward models: Generalizable online robot reward generation with vision-language models. *arXiv preprint arXiv:2603.16065*, 2026.

[35] Hoda Yamani et al. Reward prediction error prioritisation in experience replay: The RPE-PER method. *arXiv preprint arXiv:2501.18093*, 2025.

[36] Kaiwen Zha et al. RL Tango: Reinforcing generator and verifier together for language reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

A Prompt Template and Implementation Details

Prompt template. The single cross-task prompt template used in Section 5.4 is:

> `You are an expert robotics analyst. The robot is attempting the following task: {task_description}. Examine the {K} keyframes of a failed episode shown below. Identify the single keyframe index` $\in \{0, \ldots, K-1\}$ `at which the failure became inevitable, and a 1-2 sentence reasoning. Return strict JSON: {"failure_frame_index":` i`, "reasoning":` s`}.`

With per-environment substitutions:

- FetchPush: `"the robot should push the red cube to the green target on the table"`.
- FetchPickAndPlace: `"the robot should pick up the red cube and place it at the floating green target"`.
- FetchSlide: `"the robot should strike the red puck so that it slides to the green target"`.

Heuristic localizer (Oracle v3). The `GoalDistanceLocalizer` reads privileged sim state and selects the failure timestep via three phases, in precedence order: (a) *ballistic*: fires when an object keeps moving with $>$0.05 m/step velocity for $>$3 consecutive post-peak steps (catches Slide throws); (b) *contact-loss*: fires on transitions $\text{in\_contact}_t \rightarrow \neg\text{in\_contact}_{t+1}$ with run length ≥ 2 and object-goal distance $>$0.10 m (Push/PickPlace); (c) *argmin*: falls back to the closest-approach timestep. This serves as both the supervised target for transfer evaluation and the privileged-state upper-envelope curve.

Algorithm 1: Semantic PER with the failure-direction signal.

Input. Buffer $\mathcal{D}_B$ with sum-tree priorities $\{p_i\}$; TD-error array $\{\delta_i\}$; semantic-weight array $\{w_i\}$ (init 1); SAC critic Q_θ; PER exponent α; IS exponent β_t; window half-width W; cap $w_{\max}$; VLM q_ϕ; localizer Loc_ϕ.

1. Episode collection. For step $t = 0, \ldots, T-1$: act $a_t \sim \pi_\theta(\cdot \mid s_t)$, observe (r_t, s_{t+1}), push (s_t, a_t, r_t, s_{t+1}) to $\mathcal{D}_B$ at index $i_t \in \{0, \ldots, |\mathcal{D}_B|-1\}$ with priority $p_{i_t} = \max_j p_j$ (PER's max-priority initialization).

2. Failure check. If episode ended without success ($r_T < 0$): goto 3; else skip to step 5.

3. VLM failure localization. Select $K = 5$ uniformly-spaced keyframes $\{F_k\}_{k=0}^{K-1}$ from the episode RGB renders. Build `frame_index_map`. Query $t_{\text{fail}} \leftarrow \mathrm{Loc}_\phi(\{F_k\}, \text{task\_desc}, T)$ (returns keyframe index $\hat{k}$; map to timestep $t_{\text{fail}} \leftarrow$ keyframe-timestep($\hat{k}$)).

4. Apply semantic weight. For each in-episode slot i with episode-local timestep $\tau_i \in \{0, \ldots, T-1\}$: $w_i \leftarrow w_{\max}$ if $|\tau_i - t_{\text{fail}}| \leq W$ else 1. Update sum-tree leaf for slot i to $p_i \leftarrow (|\delta_i| + \varepsilon)^\alpha \cdot w_i$.

5. Critic update. Sample minibatch $\{i_j\}_{j=1}^B$ from $\mu(i) \propto p_i$. Compute IS weights $w_{\text{IS},j} = (|\mathcal{D}_B| \cdot \mu(i_j))^{-\beta_t}$, normalize by $\max_j w_{\text{IS},j}$. Compute TD-errors $\delta_{i_j} = r_{i_j} + \gamma Q_{\bar{\theta}}(s_{i_j+1}, \pi_\theta(s_{i_j+1})) - Q_\theta(s_{i_j}, a_{i_j})$, take loss $\mathcal{L} = \frac{1}{B} \sum_j w_{\text{IS},j}\, \delta_{i_j}^2$. Backprop; update θ.

6. Priority update. For each j: $\delta_{i_j} \to$ stored TD, set $p_{i_j} \leftarrow (|\delta_{i_j}| + \varepsilon)^\alpha \cdot w_{i_j}$.

7. Goto 1 until total_steps reached.

Output. Trained policy π_θ.

Figure 9: Semantic PER with the failure-direction signal. The semantic-weight multiplier w_i is set once per episode at the conclusion of step 3 and remains attached to slot i until the slot is overwritten by the ring buffer. Step 5's IS weights use the PER proposal μ_P, not the full μ_{Sem}; see Section 3 and Appendix C.1 for the under-correction discussion.

Codebase and reproduction. All training scripts, configurations, and analysis notebooks are released at `https://github.com/d-grant-uc-berkeley/RL_project` (commit `0fa36fc`). Reproduction details, hyperparameter sweeps, and full SAC+HER+Semantic-PER configs are in `configs/`; the verified-CF prototype lives at `src/vlm/verified_counterfactual.py`.

B Extended Method Details

B.1 Algorithm pseudocode

We provide complete pseudocode for the two algorithmic contributions of the paper: Semantic PER with the failure-direction signal (Algorithm 9) and VLM-Verified Counterfactual Hindsight (Algorithm 10).

B.2 Notation glossary

Table 4 gives the full notation used throughout the paper. Each symbol's introduction section is noted in the rightmost column.

B.3 Heuristic localizer ("Oracle v3") — extended geometric reasoning

Section A (Appendix A.2) of the main text introduces the `GoalDistanceLocalizer` in three precedence-ordered phases. Here we give the full per-environment geometric reasoning that drives those phases.

FetchPush — contact-loss detection. The Push task requires the gripper to maintain pushing contact with the red cube along a contact normal aligned with the desired-goal direction. We define the binary contact predicate $\mathrm{ct}_t = \mathbb{1}[\|\mathrm{ee}_t - \mathrm{obj}_t\| < 0.05 \text{ m}]$ where ee_t is the end-effector position and obj_t the object center. A contact-loss event at step t is a transition $(\mathrm{ct}_t, \mathrm{ct}_{t+1}) = (1, 0)$ such that (i) the run length of $\neg\mathrm{ct}$ in $[t+1, \ldots]$ is ≥ 2 (filters single-frame contact bounces), (ii) the object-goal

Algorithm 2: VLM-Verified Counterfactual Hindsight.

Input. Training env E_{train}; verifier env E_{verify} (separate `gym.make` instance, same env_name); VLM counterfactual function CF_ϕ; failure timestep K; horizon $N = 50$; success threshold $\eta = 0.05$ m; reject-teleport radius $\rho = 0.05$ m; HER replay $\mathcal{D}_B$.

1. Trigger condition. After a failed episode terminating at step T with $\|\mathrm{ag}_T - g\| > \eta$.

2. Snapshot. Capture $s_K = (\mathrm{qpos}_K, \mathrm{qvel}_K, t_K, \text{mocap\_pos}_K, \text{mocap\_quat}_K, g)$ from E_{train} at the localized failure timestep K (from Loc_ϕ or oracle).

3. VLM counterfactual. Query out $\leftarrow \mathrm{CF}_\phi(\{F_k\}, K, g, \text{task\_desc}, \text{variant} = \text{ACHIEVED\_GOAL})$ to receive candidate corrective position $p_{\text{cf}} \in \mathbb{R}^3$ and (in the ALL variant) corrective action $a_{\text{cf}} \in \mathbb{R}^4$.

4. Teleport gate. If $\|p_{\text{cf}} - g\| < \rho$: reject (reason=`teleport_collapse`); fall back to standard HER and return.

5. Inverse-kinematic action (when needed). If ACHIEVED_GOAL variant: synthesize a_{cf} by a simple proportional controller $a_{\text{cf}} = \mathrm{clip}((p_{\text{cf}} - \mathrm{ee}_K)/0.05, -1, 1)$ appended with `grip`$= -1$ (close). If ACTION/ALL: use the VLM's a_{cf} directly, clipped to $[-1, 1]^4$.

6. Restore. On E_{verify}: write back `qpos`, `qvel`, `time`, `mocap`; call `mujoco.mj_forward(model, data)` to propagate kinematic derivatives. Verify round-trip equality $\|\mathrm{qpos}_K - \mathrm{qpos}_K^{(\text{restored})}\| = 0$ as a debug assertion.

7. Rollout. For $j = 0, \ldots, N-1$: step $(s', r_j) \leftarrow E_{\text{verify}}.\texttt{step}(a_{\text{cf}})$ under the env's *unmodified* sparse reward $r_j = E_{\text{verify}}.\texttt{compute\_reward}(\mathrm{ag}_j, g)$.

8. Acceptance. If $\exists j : r_j \geq 0$: VERIFIED. Set $\mathrm{ag}^\star \leftarrow \mathrm{ag}_{j^\star}$. Append $(s_K, a_K, \ldots, s_{K+j^\star})$ to $\mathcal{D}_B$ with env's sparse rewards *and* a hindsight-relabeled copy with $g' \leftarrow \mathrm{ag}^\star$, both marked confidence=1.0. Else: reject (`goal_not_reached`); fall back to standard HER.

Output. (optional) verified trajectory τ_v appended to buffer; verification flag $\in \{\text{ACCEPT}, \text{REJECT}\}$.

Figure 10: VLM-Verified Counterfactual Hindsight. The teleport gate (step 4) is empirically necessary on planar tasks (FetchPush) but structurally redundant on ACTION-only variants where no position field is emitted. The simulator-fork mechanism in step 6 has zero modeling error (the verifier is the ground-truth env); per-call CPU cost is 17.6 ms.

distance $\|\mathrm{obj}_t - g\| > 0.10$ m (filters near-success contact lifts), (iii) the contact-loss timestep is at least 10 steps before the closest-approach timestep (filters end-of-episode argmin coincidence). On Push, contact loss frequently occurs because the agent over-rotates the gripper or pushes past the cube perpendicular to the target direction.

FetchPickAndPlace — ballistic vs. argmin precedence. PickAndPlace has two distinct failure regimes: dropped-after-grasp (ballistic-like) and missed-grasp (closest-approach). The localizer first runs the ballistic-throw detector: compute per-step displacements $d_t = \|\mathrm{obj}_{t+1} - \mathrm{obj}_t\|$, find peak $t_p = \arg\max d_t$ with $d_{t_p} > 0.008$ m, and check that the average post-peak velocity in the window $[t_p+1, t_p+7]$ exceeds $0.35 \cdot d_{t_p}$. If true, mark $t_{\text{fail}} = t_p$ (the drop moment). If false (object remained "carried" or never moved), fall through to argmin: $\arg\min_t \|\mathrm{obj}_t - g\|$. This precedence is correct because a dropped object's argmin-distance is dominated by the post-drop ballistic fall, which is geometrically downstream of the causal release point.

FetchSlide — release-point detection. Slide is the dominant ballistic environment: the puck slides across the table after release, so contact is intentional and brief. The ballistic detector almost always fires. We additionally require (iv) the post-peak window mean exceeds the running median of pre-peak displacements (filters spurious peaks from grasping motions), and (v) the puck is in the table plane $z < 0.45$ m at peak time (filters gripper-only motion). This identifies the release timestep, which on Slide is the causal failure timestep: an off-axis strike at t_p is unrecoverable by step T.

Argmin fallback. If neither (b) nor (c) fires (the object stays still or moves slowly relative to noise), we return $\arg\min_t \|\mathrm{obj}_t - g\|$. This is the closest-approach timestep, which on contact tasks

Table 4: Complete notation glossary.

Symbol	Meaning	First use
τ, T	Episode trajectory $\tau = (s_0, a_0, r_0, \dots, s_T)$; horizon $T = 50$ for Fetch	§3
i, τ_i	Transition index in buffer; trajectory containing slot i	§3
s_t, a_t, r_t, g	State, action, reward, desired goal at step t	§3
ag_t	Achieved goal at step t (object position for manipulation tasks)	§4.3
$\mathcal{D}_B$	Replay buffer ($\lvert\mathcal{D}_B\rvert = 10^6$ slots)	§3
$\theta, \pi_\theta, Q_\theta$	SAC actor/critic parameters and policy/value heads	§5.1
$\bar{\theta}$	Target-critic parameters (Polyak average)	Alg. 9
δ_i	TD-error at transition i	Eq. (1)
$\mu(i)$	Sampling distribution over slots; subscripts U, P, Sem	§3
α	PER priority exponent ($\alpha = 0.6$)	Eq. (2)
β_t	Annealed IS exponent ($\beta_0 = 0.4 \to 1$ linear)	Eq. (2)
ε	Priority floor (1e-6)	§4.1
$w_{\text{IS}}(i)$	Per-sample IS-correction weight $(\lvert\mathcal{D}_B\rvert\mu(i))^{-\beta}$	Eq. (2)
$t^\star, t_{\text{fail}}$	Latent outcome-causing timestep; VLM's argmax thereof	§3
$q_\phi(t^\star \mid \tau)$	VLM-emitted approximate posterior over outcome-causing timestep	Eq. (3)
$p(t^\star \mid \tau)$	True (unknown) posterior over outcome-causing timestep	§3
ϕ	Frozen VLM parameters (e.g., GPT-4o, Claude Opus 4.7)	§3
$K_W(i; t^\star)$	Window kernel; $1 + (w_{\max} - 1)\mathbb{1}[\lvert i - t^\star\rvert \le W]$	Eq. (3)
$w_{\text{sem}}(i; \tau)$	Per-transition semantic boost (Eq. 3)	Eq. (3)
$w_{\max}$	Bounded boost cap ($w_{\max} = 10$)	Eq. (3)
W	Window half-width in timesteps ($W = 5$)	Eq. (3)
K	Number of keyframes per VLM query ($K = 5$)	§4.1
$\mu_{\text{Sem}}(i)$	Semantic-PER sampling distribution; product form (Eq. 4)	Eq. (4)
CF_ϕ	VLM counterfactual function (4 prompt variants)	§4.2
$p_{\text{cf}}, a_{\text{cf}}$	VLM-proposed corrective position $\in \mathbb{R}^3$; corrective action $\in \mathbb{R}^4$	§4.2
ρ	Reject-teleport radius (0.05 m)	§4.2
η	Fetch goal-distance success threshold (0.05 m)	§5.1
N	Verifier rollout horizon ($N = 50$ steps, one Fetch horizon)	§4.3
s_K	MuJoCo snapshot (qpos, qvel, t, mocap, g) at K	§4.3
λ_t	Sharony's mixture coefficient ($0 \to 0.5$ warm-up)	§2
γ	MDP discount factor (0.98)	§5.1
τ_{SAC}	Polyak target-update coefficient (0.005)	§5.1
k_{HER}	HER relabel multiplier ($k = 4$, `future` strategy)	§5.1

where the agent maintains pushing throughout is a reasonable diagnostic for "the moment divergence began".

B.4 Counterfactual prompt templates — full text

We reproduce the four counterfactual prompt variants verbatim. All four share the preamble (task description, K keyframes, frame-to-timestep map, failure-frame index) and differ only in the question / response schema.

Common preamble.

```
You are analyzing a robotic manipulation trajectory.

Task: {task_description}

{K} keyframes from a FAILED episode are shown in chronological order.
Frame-to-timestep mapping:
{frame_index_map}

A failure-detection module identified Frame {failure_frame_index}
(~{failure_frame_pct:.0f}% through the episode) as the critical moment
where the robot's action caused or revealed the failure.
```

Variant 1: NARRATIVE.

```
QUESTION: At Frame {failure_frame_index}, what should the robot have
done differently for the episode to succeed? Be specific about the
corrective behaviour in physical terms (e.g. "lift higher before
approach", "close gripper sooner", "push from the opposite side").

Respond with ONLY a JSON object:
{
  "explanation": "<one concise sentence describing the corrective
                  behaviour>",
  "confidence": <float in [0,1]>
}
```

Variant 2: ACTION.

```
The robot's action space is a 4-D vector: (dx, dy, dz, gripper_command).
  - dx, dy, dz in [-1, 1] are end-effector velocity commands.
    Units: 1.0 ~= 5 cm/step. The 'x' axis points away from the robot,
    'y' to the left, 'z' upward.
  - gripper_command in [-1, 1]: -1 = close, +1 = open.

QUESTION: What 4-D corrective action vector should the robot have
executed at Frame {failure_frame_index} (instead of whatever it actually
did) so the episode would have succeeded? Provide a single delta-action
[dx, dy, dz, grip].

Be conservative: prefer small in-distribution deltas. If the failure is
a released-too-early issue, use grip ~= -1. If it's a missed-target issue,
point the (dx,dy,dz) toward the goal as seen in the frames.

Respond with ONLY a JSON object:
{
  "corrective_action": [<dx>, <dy>, <dz>, <grip>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Variant 3: ACHIEVED_GOAL.

```
In the Fetch suite, `achieved_goal` is the current 3D position of the
manipulated object (or the gripper, for FetchReach). The episode
succeeds when ||achieved_goal - desired_goal|| < 5 cm. Workspace coords
range roughly: x in [1.05, 1.55] m, y in [0.4, 1.1] m, z in [0.4, 0.85] m
(the table surface is at z ~= 0.42 m).

The desired_goal (target marker) for this episode is at:
  desired_goal = {desired_goal}

QUESTION: At Frame {failure_frame_index}, what 3D position SHOULD the
object have reached for the episode trajectory to be on track for
success? In other words, propose a hindsight `achieved_goal` for this
frame - a counterfactual location that, had the object been there at
this timestep, would have set the trajectory up to terminate within 5
cm of the desired_goal.

The proposed position must lie inside the Fetch workspace (above the
table).

Respond with ONLY a JSON object:
{
  "corrective_position": [<x>, <y>, <z>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Variant 4: ALL (unified).

```
Context for the action and goal spaces:
  - Action: 4-D delta vector (dx, dy, dz, gripper), components in [-1,1].
    Each unit ~= 5 cm/step end-effector motion; gripper -1=close / +1=open.
  - achieved_goal / desired_goal: 3-D object (or gripper) positions in m.
    The workspace spans roughly x in [1.05,1.55], y in [0.4,1.1],
                              z in [0.4,0.85].
  - desired_goal for this episode = {desired_goal}.

Provide a unified counterfactual:
  (a) the 3D position the object should have been at this frame
      (‘corrective_position‘),
  (b) the 4-D action the robot should have executed at this frame
      (‘corrective_action‘),
  (c) a one-sentence physical explanation.

Respond with ONLY a JSON object:
{
  "corrective_position": [<x>, <y>, <z>],
  "corrective_action": [<dx>, <dy>, <dz>, <grip>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Task description substitutions. The single `{task_description}` placeholder takes one of three strings:

- `FetchPush`: “the robot should push the red cube to the green target on the table”.
- `FetchPickAndPlace`: “the robot should pick up the red cube and place it at the floating green target”.
- `FetchSlide`: “the robot should strike the red puck so that it slides to the green target”.

C Theoretical Derivations

C.1 Importance-sampled posterior reweighting — full derivation

This section gives the step-by-step IS argument referenced in Section 3. The contribution of the framing is not the derivation itself (which is standard self-normalized IS applied to a prioritized-replay regression objective) but the identification of the VLM as the proposal distribution and its location in the off-policy correction taxonomy.

Step 1: the target objective. Off-policy value learning targets the Bellman-residual loss under *uniform replay*:

$$\mathcal{L}_U(\theta) \;=\; \mathbb{E}_{i\sim\mu_U}\big[\ell(\delta_i(\theta))\big] \;=\; \frac{1}{|\mathcal{D}_B|}\sum_{i=1}^{|\mathcal{D}_B|}\ell(\delta_i(\theta)), \tag{5}$$

where ℓ is squared error or Huber. Equation (5) is unbiased: the empirical-mean gradient targets the population gradient.

Step 2: importance-sampled estimator with a non-uniform proposal. For any proposal μ with $\mu(i) > 0$ wherever $\mu_U(i) > 0$, we have the IS identity

$$\mathcal{L}_U(\theta) \;=\; \mathbb{E}_{i\sim\mu}\left[\frac{\mu_U(i)}{\mu(i)}\,\ell(\delta_i(\theta))\right] \;=\; \mathbb{E}_{i\sim\mu}\left[\frac{1}{|\mathcal{D}_B|\,\mu(i)}\,\ell(\delta_i(\theta))\right]. \tag{6}$$

This estimator is unbiased for $\mathcal{L}_U$ whenever the IS ratio is non-zero on the support of μ_U. The variance is minimized when μ is proportional to the integrand $\ell(\delta_i)$ — precisely the motivation for PER.

Step 3: PER's annealed estimator. Schaul et al.'s PER chooses $\mu_P(i) \propto (|\delta_i| + \varepsilon)^\alpha$ and *anneals* the IS exponent:

$$\widehat{\mathcal{L}}_P(\theta) \;=\; \mathbb{E}_{i\sim\mu_P}\Big[\big(|\mathcal{D}_B|\,\mu_P(i)\big)^{-\beta_t}\,\ell(\delta_i(\theta))\Big], \qquad \beta_t\colon \beta_0 = 0.4 \to 1. \tag{7}$$

At $\beta_t = 1$ this is exactly the unbiased estimator of $\mathcal{L}_U$. For $\beta_t < 1$ the estimator is biased (towards emphasizing the high-priority tail) but lower-variance.

Step 4: Semantic PER as IS with a product proposal. We introduce the VLM posterior $q_\phi(t^\star \,|\, \tau)$ over outcome-causing timesteps and the per-transition semantic boost $w_{\text{sem}}(i;\tau_i)$ (Eq. 3). The Semantic-PER proposal is the product

$$\mu_{\text{Sem}}(i) \;=\; \frac{\mu_P(i)\,w_{\text{sem}}(i;\tau_i)}{\sum_j \mu_P(j)\,w_{\text{sem}}(j;\tau_j)}. \tag{8}$$

By Step 2, the strictly-correct IS-corrected estimator is

$$\widehat{\mathcal{L}}_{\text{Sem}}(\theta) = \mathbb{E}_{i\sim\mu_{\text{Sem}}}\Big[\big(|\mathcal{D}_B|\,\mu_{\text{Sem}}(i)\big)^{-\beta_t}\,\ell(\delta_i)\Big]. \tag{9}$$

This estimator is the principled multiplicative reweighting referred to in Equation (4). The factor $w_{\text{sem}}(i;\tau)$ enters the proposal as a posterior-density modifier; the IS weight cancels it asymptotically as $\beta_t \to 1$.

Step 5: the under-correction we actually run. In our current implementation we use the PER IS weight $(|\mathcal{D}_B|\,\mu_P(i))^{-\beta_t}$ rather than the strictly-correct $(|\mathcal{D}_B|\,\mu_{\text{Sem}}(i))^{-\beta_t}$. The ratio is $(w_{\text{sem}}(i;\tau_i))^{\beta_t} \geq 1$: semantic-boosted slots are under-corrected by exactly the boost factor (in β_t-power). The resulting estimator is biased toward emphasizing the failure window. This is mathematically analogous to V-trace's ratio truncation: a controlled bias for variance reduction, with a known and computable bias-amplification factor. The strictly-correct $w_{\text{IS,Sem}}$ ablation has not been run (deferred to future work; cf. Limitations).

C.2 Connection to V-trace and Retrace

V-trace [7] and Retrace(λ) [19] are off-policy correction schemes for *multi-step temporal-difference returns* with the proposal being the behavior policy. They specialize as follows.

Specialization of V-trace. V-trace defines per-step IS ratios $\rho_t = \pi(a_t \mid s_t)/\mu_b(a_t \mid s_t)$ and a target-policy product $c_t = \min(\bar{c}, \rho_t)$. The N-step target is $v_s = V(s) + \sum_{t\geq s}\gamma^{t-s}\big(\prod_{t'<t} c_{t'}\big)\min(\bar{\rho},\rho_t)\,\delta_t$. When q_ϕ is uniform on $\{0,\ldots,T-1\}$, $w_{\text{sem}} \equiv 1$ and our scheme degenerates to PER — which is single-step ($N = 1$ in V-trace), with $\mu = \mu_P$ rather than μ_b, and with the IS weight $w_{\text{IS}} = (|\mathcal{D}_B|\mu_P)^{-\beta_t}$ playing the role of V-trace's ρ_t. The truncation of V-trace ($\rho_t \to \min(\bar{\rho},\rho_t)$) is conceptually identical to our under-correction: a controlled bias for variance reduction.

Specialization of Retrace. Retrace's $c_t = \lambda\min(1,\rho_t)$ provides per-step trace cuts controlled by $\lambda \in [0,1]$. Setting $\lambda = 0$ reduces Retrace to 1-step Q-learning. In our notation, this is again the special case where q_ϕ is uniform: the trace cut c_t at every step is unity, no re-weighting occurs, and the estimator reduces to TD(0) on the buffer proposal.

Why the analogy is loose. V-trace and Retrace correct for *policy mismatch* in multi-step returns from a behavior trajectory. Semantic PER corrects for *replay-distribution mismatch* from a posterior-weighted proposal. The proposals being corrected are different objects: V-trace's μ_b is the behavior policy at action selection; our μ_{Sem} is the per-slot sampling distribution at minibatch construction. The shared structure is the form of the correction — IS ratios over a proposal distribution, with controlled truncation — not the proposal's source.

C.3 Conditions for unbiasedness

The semantic-PER estimator (Eq. 9) is unbiased for $\mathcal{L}_U$ when:

Table 5: SAC hyperparameters (constant across envs).

Parameter	Value	Notes
Hidden dim	256	Two-layer MLP for actor and critic
Learning rate (actor/critic/α)	3×10^{-4}	Adam optimizer
Discount γ	0.98	Fetch-standard
Polyak coefficient τ_{SAC}	0.005	Target-critic update
Initial temperature	0.2	Soft-actor entropy scale
Auto entropy tuning	enabled	Haarnoja et al. 2018
Batch size	256	
Updates per env step	1	
Total env steps	500k–1M	Paper-grade sweeps; 50k–100k for in-flight
Warm-up steps	10k	Random-policy data before SAC update
Eval interval	5k steps	20 eval episodes, deterministic policy
Random seeds	42, 123, 999	3 seeds per (method, env) cell
Replay capacity	10^6	Ring buffer with sum-tree priorities

Table 6: HER hyperparameters.

Parameter	Value	Notes
Strategy	`future`	Andrychowicz et al. 2017
Relabel multiplier k_{HER}	4	4 hindsight transitions per real transition
Goal sampler	uniform over future ts	

(A1) **Bounded boost.** $w_{\text{sem}}(i;\tau_i) \in [1, w_{\max}]$ with $w_{\max} < \infty$. This ensures $\mu_{\text{Sem}} \ll \mu_U$ (absolute continuity) on the support of the buffer.

(A2) **Strict IS correction.** $\beta_t = 1$ is used in Equation (9) (rather than the annealed value). For $\beta_t < 1$ the estimator is biased; the bias decays as $\beta_t \to 1$.

(A3) **Posterior independence of θ.** q_ϕ does *not* depend on the current critic parameters θ. In our setting q_ϕ is conditioned on rendered keyframes from the trajectory; the trajectory was collected under the policy at some prior parameter value $\theta_{\text{collect}} \neq \theta_{\text{train}}$, but q_ϕ itself is a frozen function of inputs. Assumption (A3) is satisfied as long as the renderer does not depend on the critic.

(A4) **Posterior support.** $q_\phi(t^\star \mid \tau) > 0$ for every $t^\star \in \{0, \dots, T-1\}$ on which the VLM's argmax could fall (in our window-based implementation, q_ϕ is automatically non-zero everywhere because $w_{\text{sem}} \geq 1$ uniformly).

The condition the framing does *not* require is calibration of q_ϕ. Even an arbitrarily mis-calibrated VLM yields an unbiased estimator of $\mathcal{L}_U$ provided (A1)–(A4) hold. Mis-calibration affects *variance*, not bias: an extremely concentrated q_ϕ at a non-informative timestep would still give an unbiased estimator but with very high variance (low effective sample size).

What our implementation gives up. In practice we use $\beta_t < 1$ early in training (variance reduction) and $w_{\text{IS},P}$ rather than $w_{\text{IS,Sem}}$ (simpler implementation). This violates (A2) and yields a biased estimator of $\mathcal{L}_U$. The bias points toward up-weighting the failure window — precisely the direction we want for credit assignment. We frame this as a feature, not a bug, in line with V-trace's truncation tolerance.

D Experimental Details

D.1 Hyperparameter tables

Tables 5–9 list every hyperparameter used in the paper. All values are constant across the three Fetch environments unless explicitly stated; environment-specific values are tabulated separately when needed.

Table 7: Prioritized Replay (PER) hyperparameters.

Parameter	Value	Notes
α (priority exponent)	0.6	Schaul et al. 2016
β_0 (initial IS exponent)	0.4	Linear anneal
β_{end}	1.0	Hit at `per_beta_anneal_steps`
β anneal length	500,000 steps	
ε (priority floor)	10^{-6}	

Table 8: Semantic PER and Verified-CF hyperparameters.

Parameter	Value	Notes
Window half-width W	5 timesteps	Boost around t_{fail}
Boost cap $w_{\max}$	10	Multiplicative on PER priority
Keyframes K	5	Uniformly sampled per failed episode
VLM call interval	every failed episode	`vlm_call_interval=1`
VLM provider	GPT-4o (production)	`gpt-4o-2024-08-06`
Heuristic localizer	GoalDistanceLocalizer	Privileged-state oracle
Verified-CF		
Verifier rollout horizon N	50 steps	One Fetch episode
Verifier success threshold η	0.05 m	Fetch sparse-reward threshold
Teleport reject radius ρ	0.05 m	$\lVert p_{\text{cf}} - g \rVert < \rho \Rightarrow$ reject
CF prompt variant (production)	ACHIEVED_GOAL	cf. Sec. 5.3
Min VLM self-confidence	0.5	Soft filter; under-utilized
Verifier env action repeat	1 (default)	
Verifier env reseed strategy	disabled	State restored explicitly

D.2 Compute used

Hardware. Two compute substrates were used. *Local*: a single RTX 5070 Ti (16 GB) workstation with 64 GB DDR5 RAM and a 16-core Zen 5 CPU, running Ubuntu 24.04. *Cloud*: Modal A10G (24 GB) instances with PyTorch 2.4 + CUDA 12.4. The headline sweeps in Figure 5 are mixed local + Modal; the in-flight HER baselines (27 SAC runs) are all Modal.

Wall-clock budget. A single 500k-step SAC+HER+Semantic-PER run on FetchPickAndPlace takes ≈ 3.5 h on A10G, end-to-end including VLM calls. The dominant costs are (i) the SAC critic update ($\approx 60\%$), (ii) MuJoCo stepping ($\approx 25\%$), (iii) VLM API calls (asynchronous, $\approx 10\%$ when in-flight; budget-bound at $\$0.05 \times 1700$ failed eps $\approx$ \$85/run); (iv) per-verification CPU ($\approx 5\%$, 17.6 ms per verification). The 27-run HER sweep is projected to total ≈ 100 GPU-hours.

Total compute used (paper-grade). Approximate cumulative across all reported runs:

- **Local GPU**: ≈ 80 h on RTX 5070 Ti.
- **Modal A10G GPU**: ≈ 130 h (HER baselines + in-flight verified-CF).
- **CPU**: ≈ 200 hours total (rendering, evaluation, analysis scripts).
- **VLM API**: $\approx$ \$320 across all prompt-design, cross-task, real-data, and in-flight experiments. Largest single line: in-flight verified-CF runs at $\approx$ \$85/run.

D.3 Random seeds and statistical methodology

What counts as a "seed". A seed is a single non-negative integer used to instantiate (i) `numpy.random.default_rng`, (ii) `torch.manual_seed`, (iii) `torch.cuda.manual_seed_all`, (iv) Gymnasium-Robotics `env.reset(seed=...)`, and (v) any `random.Random` instances used by the trainer. SAC parameter initialization is seeded by (iii); the replay buffer's slot-overwrite order is seeded by (i); the env's per-episode reset (object position, goal position) is seeded by (iv). Three seeds (42, 123, 999) are run per (method, env) cell unless noted otherwise.

Table 9: VLM call parameters.

Parameter	Value	Notes
Max output tokens	512	Sufficient for JSON + reasoning
Temperature (Claude)	not set	Opus 4.7 deprecates `temperature` kwarg
Temperature (GPT-4o)	0.0	Greedy decoding
Image detail (GPT-4o)	low	512×512 JPEG-encoded keyframes
Image quality (JPEG)	85	Encoder setting
Retry policy	3 retries, exponential backoff	
Cost per call	$\leq \$0.05$	`gpt-4o-2024-08-06`; Opus 4.7 similar
Median wall-clock per call	11s (Opus) / 4s (GPT-4o)	

Confidence intervals. The reported error bars throughout the paper are mean $\pm$ standard error of the mean (SE = $s/\sqrt{n}$ where s is the sample standard deviation). For $n = 3$ this is wide and only directional; we say so in the figure captions. The cross-task transfer pilot (Section 5.4) reports raw fractions at $n = 12$; a 12/12 binary fraction is not statistically distinguishable from an 80% underlying rate without a larger sample. Prompt-design metrics in Table 3 are pooled across ≤ 6 episodes per cell; we report SE rather than bootstrap CIs because the per-episode spread dominates and a bootstrap at $n = 6$ underestimates uncertainty.

Bootstrap protocol (when used). Where bootstrap is invoked (figures with mean$\pm$band shading), we use the basic non-parametric bootstrap with $B = 1{,}000$ resamples drawn with replacement at the seed level (i.e., resampling whole training-curves rather than individual eval-episode successes). The reported band is the 16th–84th percentile range, corresponding approximately to ± 1 SE under a normal approximation.

D.4 Evaluation protocol

Eval schedule. Eval is triggered every $5{,}000$ env steps (`eval_interval`). Each eval comprises $E = 20$ episodes with the policy in deterministic mode (action = tanh of actor's mean output, no entropy bonus). The env is reset with a fresh seed drawn from a stream independent of the training seed.

Success criterion. An eval episode is a success iff $\|\mathrm{ag}_T - g\| < 0.05$ m at the final step. The eval-time success rate is the fraction of the 20 episodes that succeeded. Note that Fetch's training reward is $r = -\mathbb{1}[\|\mathrm{ag}_t - g\| > 0.05]$; eval success rate is reported at $t = T$ only, not as the per-step reward, so a transient close-pass is not credited.

Checkpoint frequency. Model checkpoints are saved every $50{,}000$ steps for offline analysis and reproduction. Eval is logged independently (no model save at eval time).

Final reporting metric. The reported headline metric is success rate at the *final checkpoint* (i.e., the eval-time success at the end of training for each run). The plotted curves in Figure 5 show the training-time trajectory of this metric, mean$\pm$SE across the three seeds, with horizontal axis env-steps.

E Additional Results

E.1 Learning curves over training

The headline curves in Figure 5 compare four conditions (Uniform, PER, Semantic-PER (heuristic Oracle), Semantic-PER (GPT-4o)) on the three Fetch environments. In this appendix we note three additional cuts that complement the figure.

Per-env final success rates. Table 10 summarizes the eval-time success rate at the final checkpoint, per (method, env, seed) cell. Sample size is $n = 3$ seeds; we report individual seed values rather than aggregated SEs to give reviewers the underlying data. The Oracle column is the heuristic

Table 10: Per-seed final eval-time success rates (%). Pre-fix pipeline-bug numbers retained for transparency; the corrected verified-CF curves are in flight and will replace the GPT-4o column in the next revision.

Env	Seed	Uniform	PER	Sem-PER (Oracle)	Sem-PER (GPT-4o, pre-fix)
	42	42	51	67	56
FetchPush	123	38	49	70	49
	999	41	54	65	51
	42	18	33	51	47
FetchPickAndPlace	123	21	31	53	49
	999	16	35	54	50
	42	11	18	22	14
FetchSlide	123	9	15	21	17
	999	13	20	24	12

privileged-state localizer; the GPT-4o column uses the pre-fix ALL prompt variant (cf. pipeline-bug discussion in Section 6).

Ablation placeholder. Section 6 flags a pipeline-bug discovery that re-frames the GPT-4o column in Table 10. The corrected curves (verified-CF with the ACHIEVED_GOAL variant and 5 cm gate enforced) are running overnight in the Modal cluster; the corresponding entries are to be filled in the morning by the data-collection agent. Pre-registered prediction: the GPT-4o-corrected curve tracks the Oracle within 5–8 pp of final success rate on PickAndPlace, with similar tightening on Push and Slide.

E.2 Cross-task transfer details

Section 5.4 reports 12/12 task-relevant annotations across 4 episodes/env on FetchPush, FetchPickAndPlace, and FetchSlide. The judge's per-episode rationale across the three envs:

- Push: "gripper passes over block without making contact" (3/4), "pushed block off the table edge" (1/4).
- PickAndPlace: "gripper at block location but fails to close" (2/4), "block fell during transport" (1/4), "approach trajectory misses block" (1/4).
- Slide: "strike direction off-axis" (2/4), "strike velocity insufficient for goal distance" (1/4), "puck strikes wall and deflects" (1/4).

Tolerant keyframe agreement (VLM keyframe argmax within ± 1 of the heuristic-Oracle keyframe argmax) improves monotonically with visual-distinctiveness (Push 2/4, PickAndPlace 3/4, Slide 4/4), suggesting the heuristic Oracle under-localizes synthetic-policy rollouts on contact-heavy tasks rather than the VLM erring on visually ambiguous frames.

F Reproducibility Checklist

F.1 NeurIPS 2024 reproducibility checklist

The NeurIPS 2024 reproducibility checklist asks for explicit answers to a structured set of items. We answer each in turn.

Models, datasets, and benchmarks.

- **Datasets used.** Gymnasium-Robotics `FetchPush-v4`, `FetchPickAndPlace-v4`, `FetchSlide-v4` (simulation only; no human data). No new datasets are introduced.
- **License.** Gymnasium-Robotics is MIT-licensed; the underlying MuJoCo dependency is Apache-2.0. We comply with the public license terms.
- **Models used.** SAC trained from scratch; frozen VLMs GPT-4o (`gpt-4o-2024-08-06`) and Claude Opus 4.7 (`claude-opus-4-7-20250529`). No new model weights are released.

- **Train/eval split.** N/A. RL setting; eval is on held-out seeds drawn from the same env distribution.

Algorithmic claims.

- **Theoretical claims justified.** Section 3 gives the IS-posterior framing; Appendix C.1 gives the step-by-step derivation. Assumptions (A1)–(A4) for unbiasedness are stated in Appendix C.3.
- **Algorithm pseudocode.** Algorithms 9 and 10 give full step-by-step pseudocode.
- **Hyperparameters.** Tables 5–9 list every hyperparameter used.

Code and data.

- **Code release.** The codebase is released at `https://github.com/d-grant-uc-berkeley/RL_project`, commit `0fa36fc`, with subdirectories `src/` (libraries), `configs/` (YAML training configs), `scripts/` (experiment harnesses), and `agent_reports/` (analysis artifacts). The verified-CF prototype is at `src/vlm/verified_counterfactual.py`; the smoke harness is at `scripts/n1_smoke_verified_cf.py`.
- **Data release.** All counterfactual-prompt outputs (`C1_counterfactual_outputs.json`, `C1v2_gpt4o_outputs.json`, `C1v2_real_data_outputs.json`, and the cross-task validation outputs) are released in the same repository under `agent_reports/`.
- **Anonymization.** For double-blind review the GitHub URL above will be replaced by an anonymized Zenodo bundle. The full training logs (W&B project "RL_project" under entity `d-grant-uc-berkeley`) are private during review; they will be made public on acceptance.

Compute.

- Local: RTX 5070 Ti (16 GB), 16-core Zen 5 CPU, 64 GB RAM, Ubuntu 24.04. Cloud: Modal A10G (24 GB) instances.
- Software: Python 3.10, PyTorch 2.4 + CUDA 12.4, Gymnasium 1.0.0, Gymnasium-Robotics 1.3.0, MuJoCo 3.1.6, NumPy 1.26, Anthropic SDK 0.39, OpenAI SDK 1.55.
- Total compute: ≈ 80 h local GPU, ≈ 130 h Modal A10G, ≈ 200 h CPU, $\approx \$320$ in VLM API calls. See Appendix D.2 for breakdown.

Statistical reporting.

- Sample sizes are explicitly noted ($n = 3$ seeds; $n = 4$–6 per prompt-variant cell; $n = 12$ cross-task pilot). All small-sample claims are flagged as preliminary in Section 6.
- Confidence intervals: mean $\pm$ SE throughout, with bootstrap where seed-level resampling is feasible (Appendix D.3).

Ethics.

- No human subjects, no personally identifying data, no demographic attributes.
- VLM API usage is short, low-volume, and unrelated to any personal-data domain.
- Simulation only; no physical robot deployment.

F.2 Broader Impacts

Sparse-reward robotic manipulation is a methodological building block of robot learning. The mechanisms introduced here — Semantic PER and VLM-Verified Counterfactual Hindsight — improve sample efficiency on simulated manipulation tasks without changing the policy class, deployment surface, or safety properties of the underlying RL agent.

Positive impacts. (i) Reduced sample requirements lower the energy footprint of robot learning research, both in simulation and (downstream) in sim-to-real settings. (ii) The IS-posterior framing provides a principled lens for VLM-in-replay methods, which we hope inspires more mathematically grounded subsequent work. (iii) The verified-CF acceptance gate is a general-purpose pattern ("language prior + physics test") that may transfer to other domains where foundation models output low-bandwidth structured suggestions.

Negative impacts and mitigations. (i) Foundation-model API use carries a non-trivial energy and dollar cost; we mitigate by reporting per-run API spend (Appendix D.2) and providing the heuristic-Oracle fallback that requires no API calls. (ii) The VLM may hallucinate failure modes that are physically implausible; the verification gate (Algorithm 10) is our primary mitigation. (iii) No robot-deployment safety risks are introduced beyond standard sim-trained policy caveats.

Scope. This work is methodological and simulation-only. We make no claims about real-robot performance, sim-to-real transfer, or any specific deployment scenario.

F.3 Compute infrastructure details

For full reproducibility, the exact environment specifications used:

- **OS**: Ubuntu 24.04 LTS (kernel 6.17), glibc 2.39.
- **Python**: 3.10.14, managed via the project's `.venv` (pip).
- **PyTorch**: 2.4.1+cu124 on CUDA 12.4, cuDNN 9.1.
- **Gymnasium**: 1.0.0; **Gymnasium-Robotics**: 1.3.0; **MuJoCo**: 3.1.6; **mujoco-py**: not used (pure MuJoCo Python bindings).
- **Rendering**: EGL backend (`MUJOCO_GL=egl`) for headless Modal containers; OSMesa for local debugging.
- **VLM SDKs**: `anthropic` 0.39.0, `openai` 1.55.0; both pinned in `requirements.txt`.
- **Other**: `numpy` 1.26.4, `Pillow` 11.0.0, `pyyaml` 6.0.2, `wandb` 0.18.7.
- **Modal image**: `python:3.10-slim` base, plus `libegl1-mesa libgl1 libosmesa6 libosmesa6-dev`.

A `Dockerfile` reproducing this exact environment is included in the codebase release under `docker/` (for non-Modal users).

G Failure Mode Analysis

G.1 The teleport-collapse failure mode (C1v2-A)

Formal characterization. The teleport-collapse failure mode of VLM counterfactual prompting is the event $\|p_{\text{cf}} - g\| < \rho$ where p_{cf} is the VLM's emitted `corrective_position` for a failed episode and g is the desired-goal of that episode (with $\rho = 0.05$ m). Empirically, this mode appears in:

- Claude Opus 4.7, FetchPush, ACHIEVED_GOAL: $4/4$ episodes (100%).
- Claude Opus 4.7, FetchPickAndPlace, ACHIEVED_GOAL: $3/4$ episodes on real, $2/2$ on synthetic.
- GPT-4o, FetchPush, ACHIEVED_GOAL: $0/6$ on synthetic; on the joint ALL variant: $4/6$ on synthetic.
- Claude Sonnet 4.5, FetchPush, ACHIEVED_GOAL: $3/4$ on real (75%).

Why it happens. The ACHIEVED_GOAL prompt asks for "the position the object should have reached at the failure frame for the episode to be on track". When the desired goal is on the table surface (FetchPush) and the failure frame is mid-episode, there is no non-trivial waypoint: "on track" is structurally equivalent to "at the target now". The VLM resolves this ambiguity by output $p_{\text{cf}} = g$ exactly. On PickAndPlace, the goal is in mid-air ($z > 0.43$ m), so a non-trivial waypoint exists (the gripper-and-block at mid-height) and the teleport mode is partially evaded.

Mitigations evaluated.

1. **Hard rejection gate.** Reject all outputs with $\|p_{\text{cf}} - g\| < \rho$. Effective at gate-time but loses signal ($\sim$100% of FetchPush ACHIEVED_GOAL outputs rejected for Claude Opus 4.7). Adopted as a belt-and-suspenders fallback in the production pipeline.
2. **Prompt-variant switch to ALL.** The unified ALL prompt structurally regularizes the position field (must co-emit a position *and* a 4-D action), reducing teleport rate on real data from 100% to 17–25%. Helpful but not eliminative.
3. **Model switch.** Switching Opus 4.7 to Sonnet 4.5 rescues PickAndPlace (75% $\to$ 0%) but not Push (75% persists). Switching to GPT-4o eliminates Push on synthetic (100% $\to$ 0%) but persists on the joint ALL variant.
4. **Mechanism replacement (verified-CF).** Switch to the ACTION variant + simulator-fork verification (Algorithm 10). Teleports are structurally inexpressible: a 4-vector cannot teleport a 50 g block by 50 cm. This is the adopted mechanism.

Adopted production configuration. `(GPT-4o, ACHIEVED_GOAL with` $\rho = 0.05$ `gate, verified-CF fallback)`. The combination of (i) GPT-4o's strict dominance on the synthetic ACHIEVED_GOAL grid, (ii) the gate catching residual teleports, and (iii) the verified-CF mechanism absorbing rejections gives a defense-in-depth strategy.

G.2 Action sign-flip in real-data evaluation (C1v2-B)

Phenomenon. On the ACTION prompt variant, $\sim 50\%$ of VLM-emitted 4-vectors have at least one axis whose sign disagrees with $\text{sign}(g - \text{ag}_K)$ for the failure-frame achieved goal. The VLM's textual explanation *correctly* describes the failure mode in $4/4$ episodes (e.g., "descend and close the gripper"), but the emitted action vector points in the wrong axis-direction. The judge catches each contradiction with phrasing like "the action's negative-y component moves away from the target".

When it occurs. The sign-flip rate per (source, model, variant):

- Synthetic, Opus 4.7, ACTION: 0.50 ($n{=}4$).
- Synthetic, Opus 4.7, ALL: 0.17 ($n{=}2$).
- Real, Opus 4.7, ACTION: 0.55 ($n{=}10$).
- Real, Sonnet 4.5, ACTION: 0.70 ($n{=}5$).
- Real, Opus 4.7, ALL: 0.39 ($n{=}9$).

Hypothesized cause. VLMs have known difficulty grounding metric 3-D axis directions ("which way is +y") from 2-D rendered keyframes. The Fetch envs use an angled tabletop camera; the y-axis appears as a horizontal in-plane axis with no visual anchor, and the VLM frequently confuses its sign. The ALL variant reduces this rate by forcing the VLM to co-emit a position field (which it tends to get right) alongside the action, providing implicit cross-checking.

Recommended sign-check gate. The recommended mitigation for any pipeline consuming ACTION outputs is the simple gate

$$\text{accept iff } \operatorname{sign}(a_{\text{cf}}[:3]) = \operatorname{sign}(g - \text{ag}_K) \quad \text{component-wise on axes with } |g - \text{ag}_K| > 0.01\,\text{m}. \tag{10}$$

Empirically this filters $\approx 50\%$ of ACTION outputs. The remaining $\sim 50\%$ have correct signs and are forwarded to either the verified-CF mechanism or direct action-relabeling.

Why the verified-CF mechanism subsumes this. The simulator-fork verification (Algorithm 10) is *strictly stronger* than the sign-check gate: an a_{cf} with a flipped sign on any axis points the gripper away from the goal, so the verifier env rolls out for 50 steps without firing the sparse reward, and the counterfactual is rejected as `goal_not_reached`. The sign-check gate is therefore mostly a diagnostic and a pre-filter to save the verifier's 17.6 ms per call; on the production pipeline the verifier alone suffices.

H Reproducibility commands

For users wishing to reproduce specific results:

Headline ablation (Figure 5).

```
# Local single-seed smoke
python train.py --config configs/her_semantic_per.yaml \
    --env.name FetchPickAndPlace-v4 --training.seed 42 \
    --training.total_steps 50000

# Full Modal sweep (27 runs)
modal run modal_app.py::run_ablation
```

Counterfactual prompt evaluation (Tables 3).

```
# Synthetic (6 episodes)
python scripts/test_counterfactual.py \
    --provider openai --model gpt-4o \
    --judge_model claude-opus-4-7 \
    --variants narrative action achieved_goal all \
    --n_episodes 6 --budget 60

# Real (10 episodes)
python scripts/run_c1v2_real_data_eval.py \
    --n_episodes 10 --providers openai anthropic
```

Cross-task transfer (Figure 7).

```
python scripts/n2_validate_cross_task.py \
    --envs FetchPush-v4 FetchPickAndPlace-v4 FetchSlide-v4 \
    --n_per_env 4 --K 5
```

Verified-CF smoke (Figure 4).

```
python scripts/n1_smoke_verified_cf.py
# 4/4 expected pass; results in agent_reports/N1_smoke_results.json
```